

Reinforcement Learning Practice - Week 2

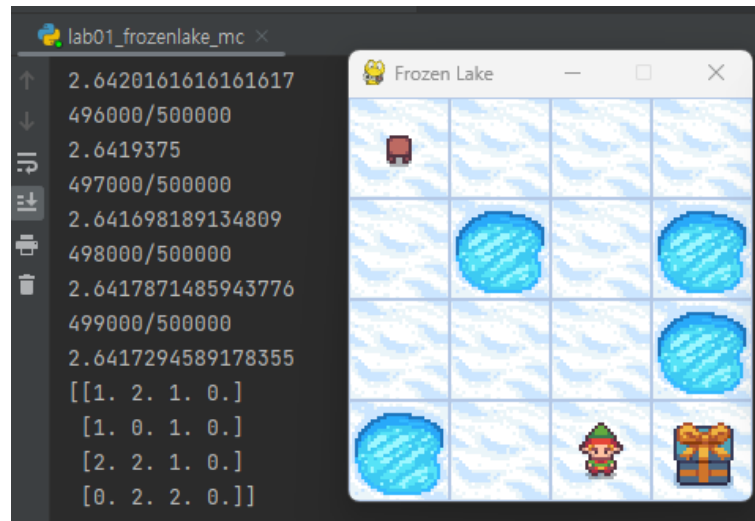
June 1, 2023

IIIXR - Juyeong Hwang

dudyyyy4@khu.ac.kr

Last Practice

- First, let's revisit the reinforcement-learning flow using Gym.
- In last week's Frozen Lake practice we: 1) reset the environment, 2) let the agent choose an action, 3) executed that action in the environment, 4) stored next_state, reward and done,
- 5) and updated the policy based on them (update_qtable).
- Looking at the result, however, the agent keeps repeating the same actions after training.

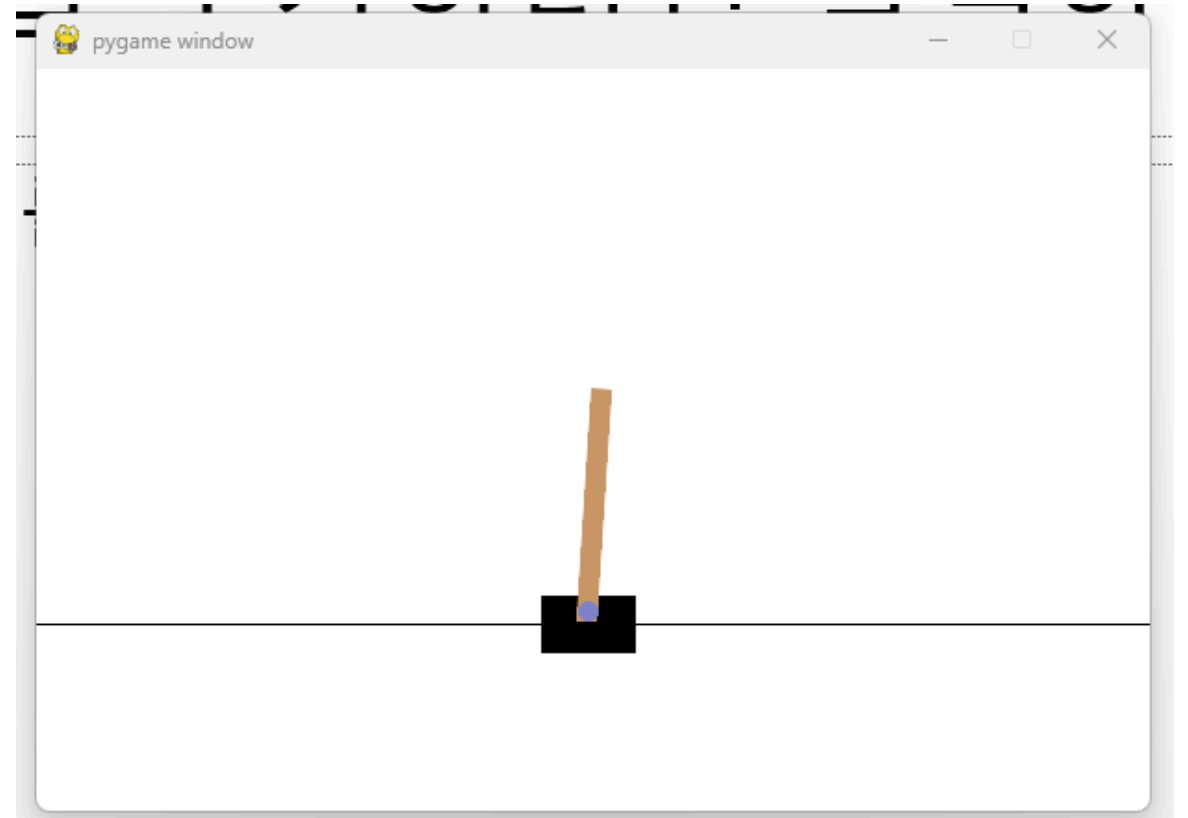
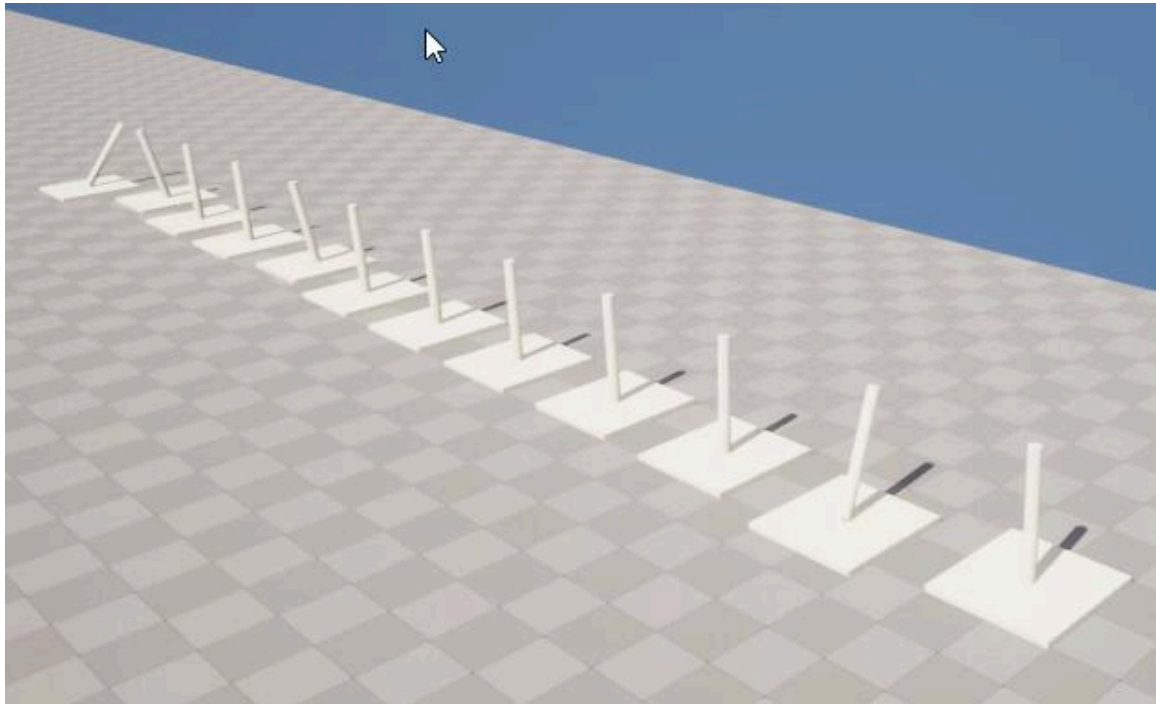


Policy-based Agent

- The previous agent moves according to the state-action value Q , so it is called a value-based agent.
- After training, $Q(s,a)$ is fixed and the agent simply picks the highest value - a deterministic way of choosing actions.
- Playing rock-paper-scissors, this agent would always throw the single most successful move.
- An agent with a stochastic policy is called a policy-based agent.
- It can hold a policy such as choosing rock/paper/scissors with $1/3$ probability each - a much more flexible policy.
- In this practice we will use the most representative model: PPO.

Practice Goal

- Build an environment for reinforcement learning inside Unreal (connect Gym and Unreal).



PPO

- Implementing PPO itself is not the goal, so the implementation details are omitted.

PPO code

- 본인의 폴더에 "SimplePPO.py"라는 이름으로 아래 내용 복사

```
: import numpy as np
import torch
import torch.nn as nn
import torch.nn.functional as F
import torch.optim as optim
from torch.distributions import Categorical

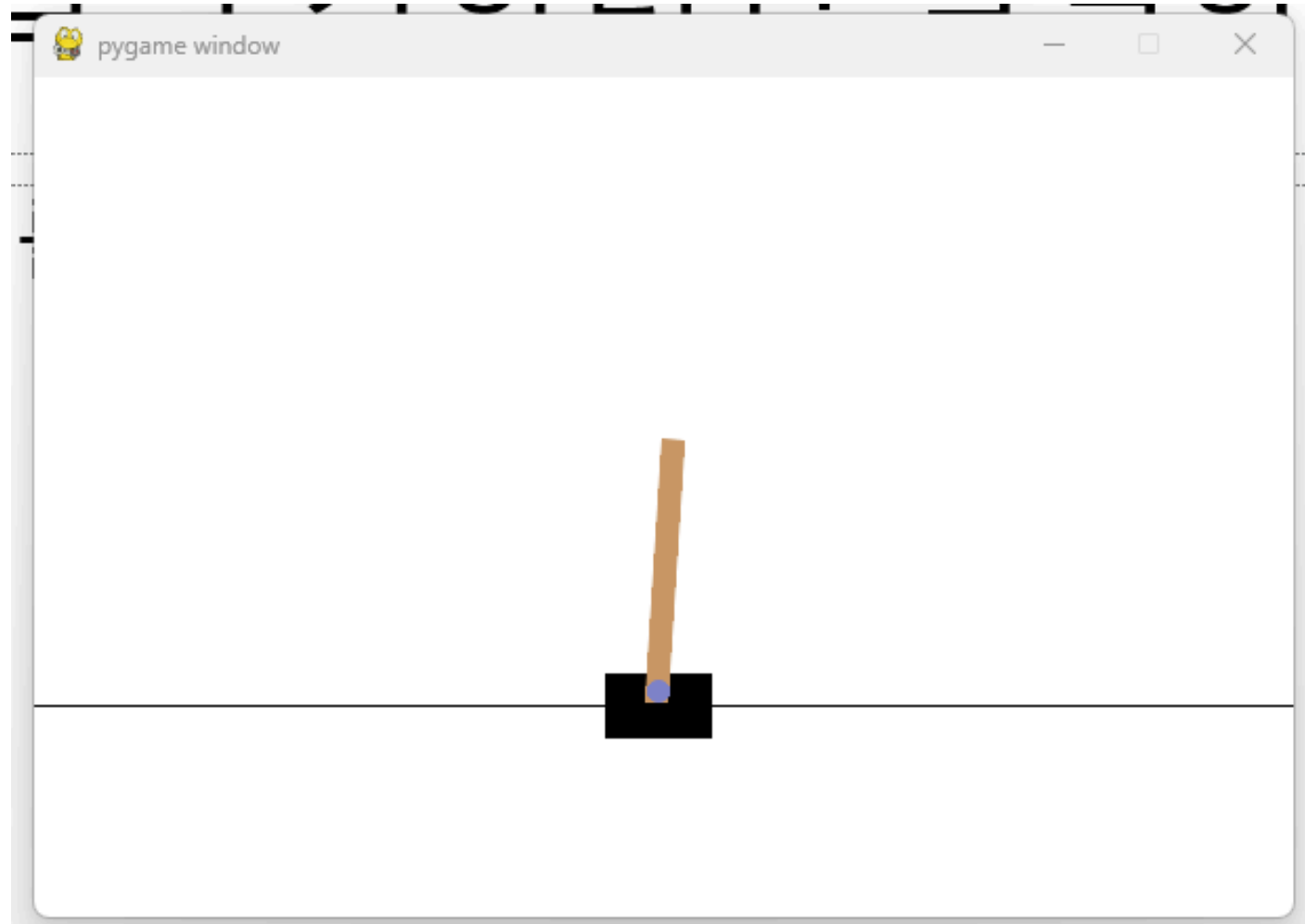
# Hyperparameters
learning_rate = 0.0005
gamma = 0.98
lmbda = 0.95
eps_clip = 0.1
K_epoch = 32

class PPO(nn.Module):
    def __init__(self, input_dim, action_dim):
        super(PPO, self).__init__()
        self.data = []

        self.fc1 = nn.Linear(input_dim, 256)
        self.fc2 = nn.Linear(256, action_dim)
```

Understanding Gym's CartPole Environment

- Action space - 0: push cart to the left / 1: push cart to the right.
- Observation space - cart position, cart velocity, pole angle, pole angular velocity.
- Gym documentation: https://gymnasium.farama.org/environments/classic_control/cart_pole/



Gym CartPole & PPO Usage

- 1) Import what we need. 2) Set up the env and the agent.

```
import gym
from SimplePPO import PPO
import matplotlib.pyplot as plt

max_episode_num = 30
env = gym.make('CartPole-v1', render_mode = 'human')
agent = PPO(input_dim=4, action_dim=2)
```

Gym CartPole & PPO Usage

- 1) Initialize the episode.
- 2) The agent selects an action from the current state - prob is the probability of each action (left / right).
- 3) Act with that action: env.step(action).

```
for ep in range(max_episode_num):  
  
    #에피소드 초기화  
    time, episode_reward, done = 0,0,False  
    state = env.reset()  
    state = state[0]  
    print(f"=={ep}==")  
    while not done:  
        prob, action = agent.select_action(state)  
        next_state, reward, done, _, _ = env.step(action)  
        env.render()
```


Gym CartPole & PPO Usage

- 1) Store the tuple obtained from one step: (state, action, reward, next state, action probability, done).
- 2) Accumulate the reward.

```
transition = (state.tolist(), action, reward, next_state.tolist(), prob[action].item(), done)
#print(transition)
agent.put_data(transition)
episode_reward += reward
```

Gym CartPole Environment

- The Gym-based CartPole environment is complete. The full code is on the side.

```
import gym
from SimplePPO import PPO
import matplotlib.pyplot as plt

max_episode_num = 30
env = gym.make('CartPole-v1', render_mode = 'human')
agent = PPO(input_dim=4, action_dim=2)

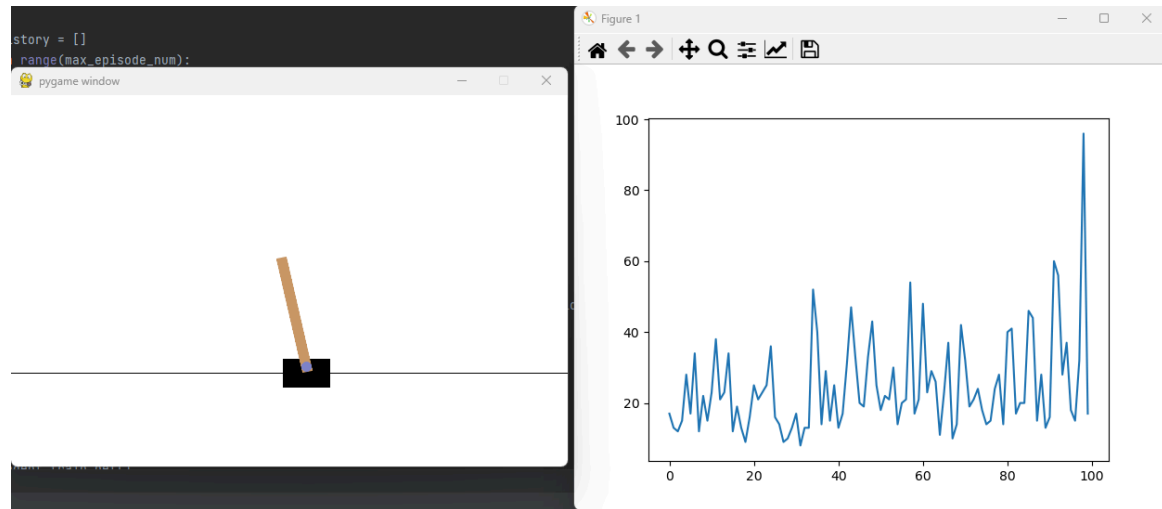
reward_history = []
for ep in range(max_episode_num):

    #에피소드 초기화
    time, episode_reward, done = 0,0,False
    state = env.reset()
    state = state[0]
    print(f"=={ep}==")
    while not done:
        prob, action = agent.select_action(state)
        next_state, reward, done, _, _ = env.step(action)
        env.render()

        transition = (state.tolist(), action, reward, next_state)
        #print(transition)
        agent.put_data(transition)
        episode_reward += reward
        state = next_state

    if(ep>0 and ep%10==0):
        agent.train_net()
        reward_history.append(episode_reward)

plt.plot(reward_history)
plt.show()
```

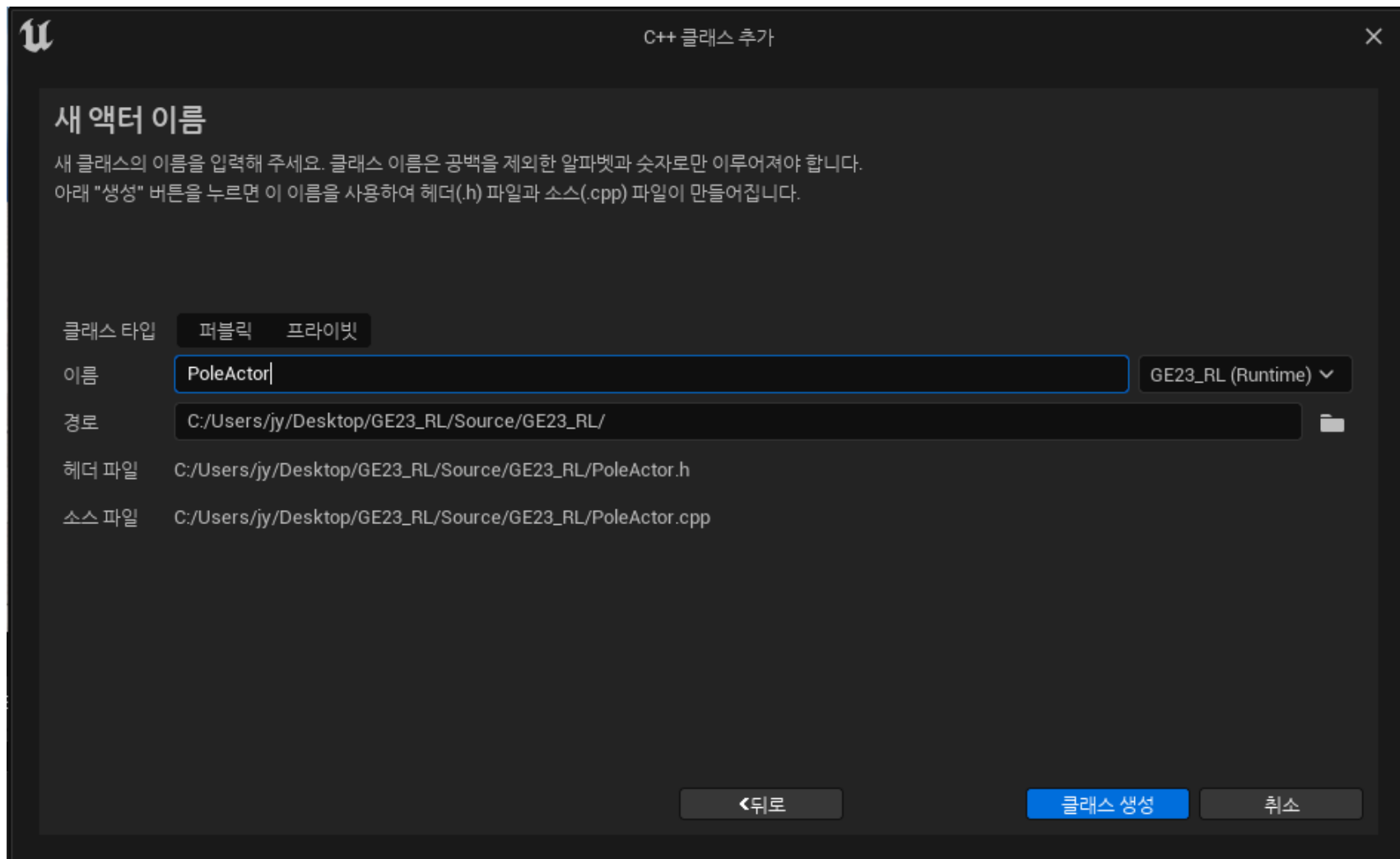


Unreal Architecture Overview

- We will now port what we just did into Unreal:
- 1) Read the cart's state from Unreal.
- 2) Pick an action with PPO in Python and send it to Unreal.
- 3) Unreal acts on it and returns the observation to Python for training.
- Python <-> Unreal: state / action / observation

Unreal - Create an Actor

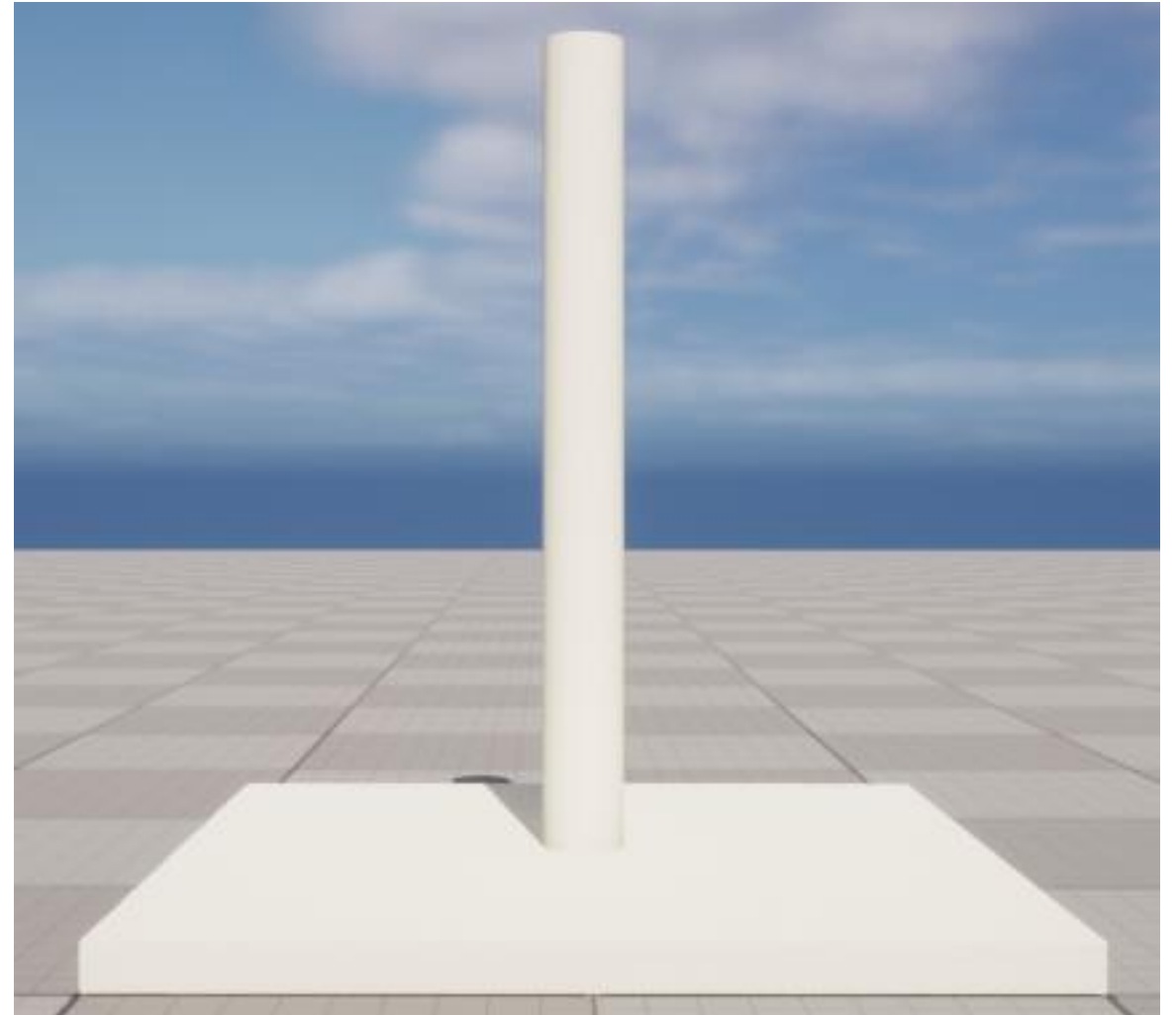
- Create a C++ class named "PoleActor".
- How: C++ Classes > GE23_RL folder > right-click empty space > New C++ Class.



PoleActor.h

- Add Base and Pole components.

```
UCLASS()  
- UE 청사진용 CodeLens  
class GE23_RL_API APoleActor : public AActor  
{  
    GENERATED_BODY()  
...  
public:  
    // Sets default values for this actor's properties  
    APoleActor();  
  
    UPROPERTY(BlueprintReadWrite, EditAnywhere)  
        UStaticMeshComponent* Base;  
  
    UPROPERTY(BlueprintReadWrite, EditAnywhere)  
        UStaticMeshComponent* Pole;  
...  
protected:  
    // Called when the game starts or when spawned  
    virtual void BeginPlay() override;
```



PoleActor.cpp

- Add the RootComponent, Base and Pole.
- Run live coding with Ctrl+Alt+F11 (the compile step we used to do in UE4).

```
#include "PoleActor.h"

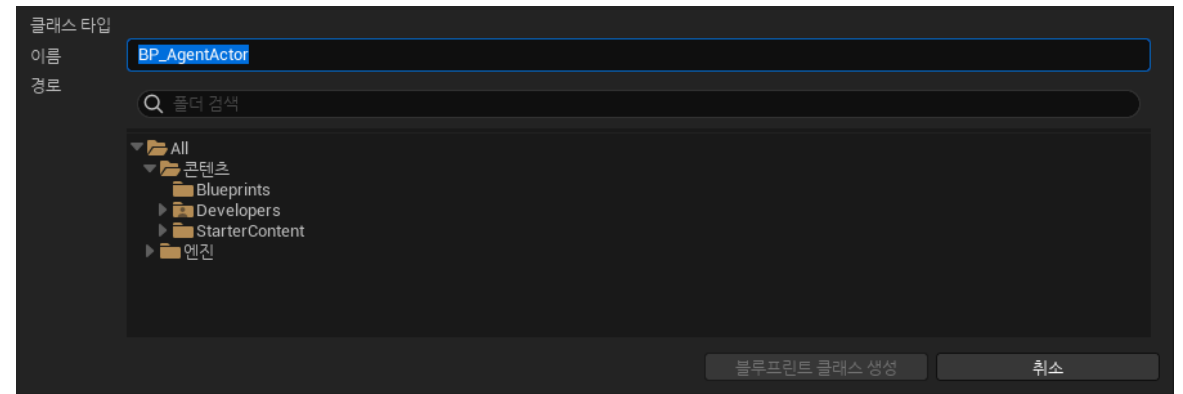
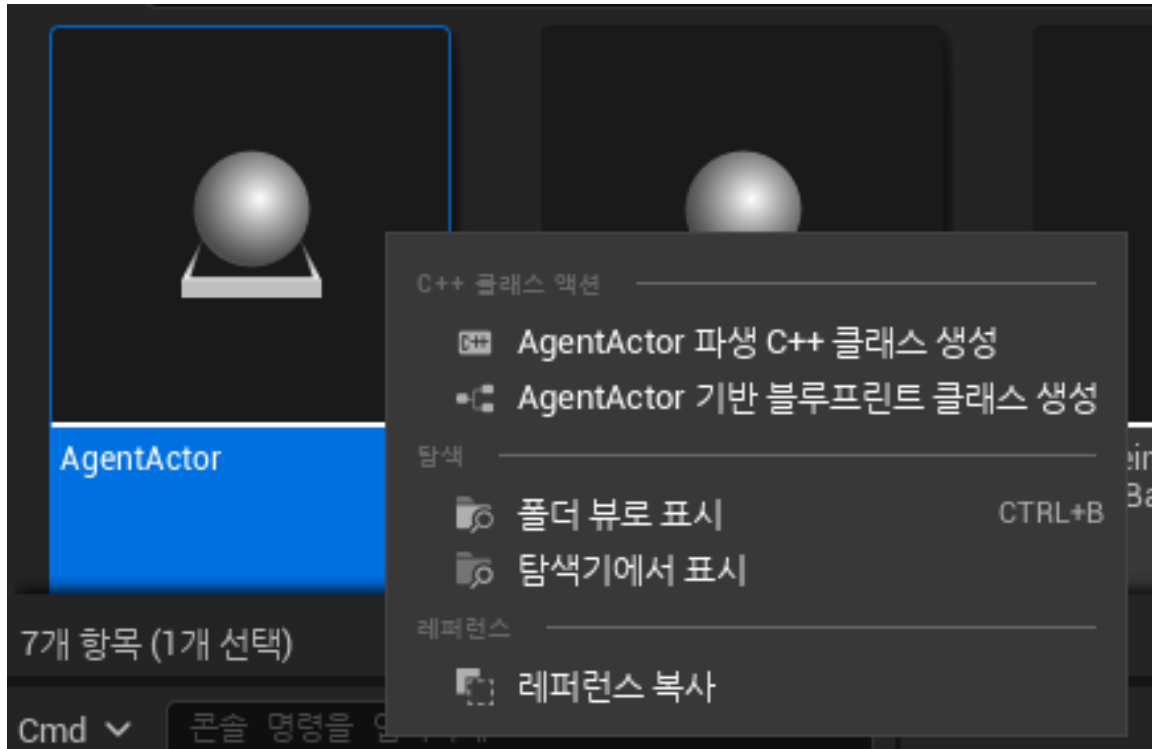
// Sets default values
APoleActor::APoleActor()
{
    // Set this actor to call Tick() every frame. You can turn this off to improve
    PrimaryActorTick.bCanEverTick = true;

    RootComponent = CreateDefaultSubobject<USceneComponent>(TEXT("RootComponent"));

    Base = CreateDefaultSubobject<UStaticMeshComponent>(TEXT("Base"));
    Pole = CreateDefaultSubobject<UStaticMeshComponent>(TEXT("Pole"));
}
```

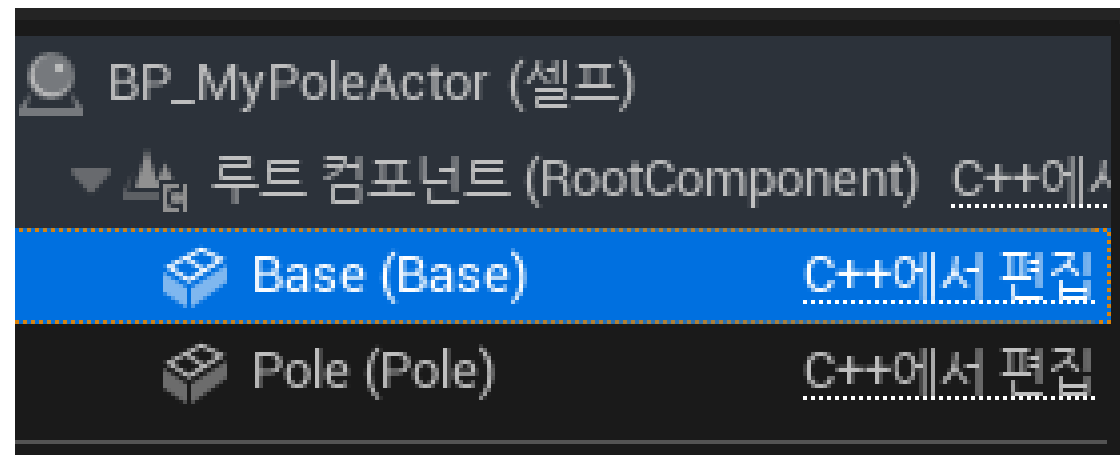
Create BP_PoleActor

- Right-click the C++ class and choose "Create Blueprint class based on PoleActor".
- Name: BP_PoleActor > Create Blueprint Class.
- (The screenshot says AgentActor, but proceed with PoleActor.)



Open BP_PoleActor

- It should look like the screenshot.



Base

▼ 트랜스폼

스케일 ▼ 🔒

2.0

2.0

0.1

↺

모빌리티

스태틱

스테이셔너리

무버블

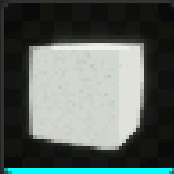
▼ 소켓

부모 소켓

📁 ✕

▼ 스태틱 메시

스태틱 메시



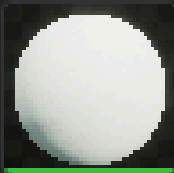
Shape_Cube ▼

↶ 📁

↺

▼ 머티리얼

엘리먼트 0



M_Basic_Wall ▼

↶ 📁 🏠 ▼

↺

컴퓨터 툴

17

Pole

▼ 피직스

피직스 시뮬레이트

☒

질량(kg)

144.134964

선형 댐핑

5.0

각 댐핑

0.0

중력 활성화

☒

▼ 컨스트레인트

위치 잠금

X☐ Y☐ Z☐

회전 잠금

X☒ Y☐ Z☒

모드

Default

방사형 임펄스 무시

☐

방사형 포스 무시

☐

데미지 시 충격량 적용

☒

피직스를 자을 프록시로 리플리케이션트

☒

▶ 고급

▼ 트랜스폼

스케일

0.2

0.2

2.0

모빌리티

스태틱스테이셔널러무버블

▼ 소켓

부모 소켓

▼ 스태틱 메시

스태틱 메시

Shape_Cylinder

▼ 재료리얼

엘리먼트 0

M_Basic_Wall

▼ 콜리전

시뮬레이션 중 히트 이벤트 생성

☐

피직스 재료리얼 오버라이드

None없음

오버랩 이벤트 생성

☒

Can Character Step Up On

Yes

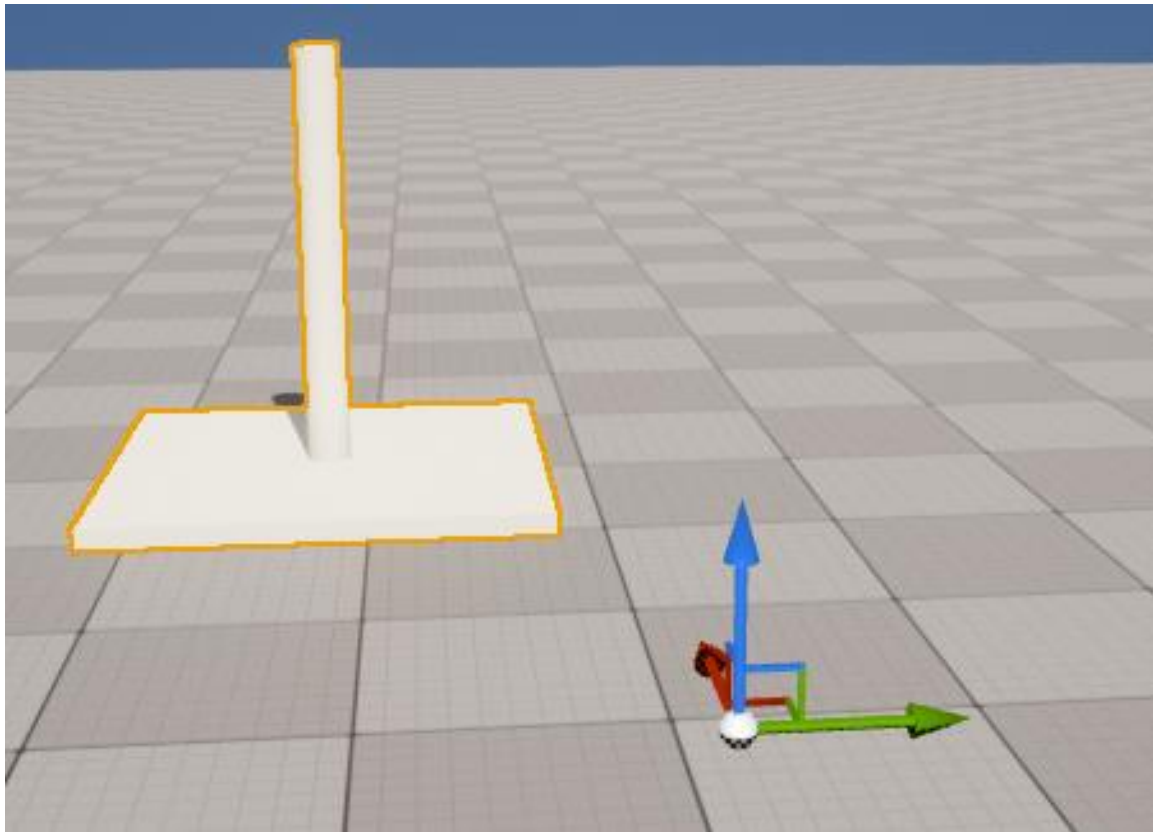
▶ 콜리전 프리셋

PhysicsActor

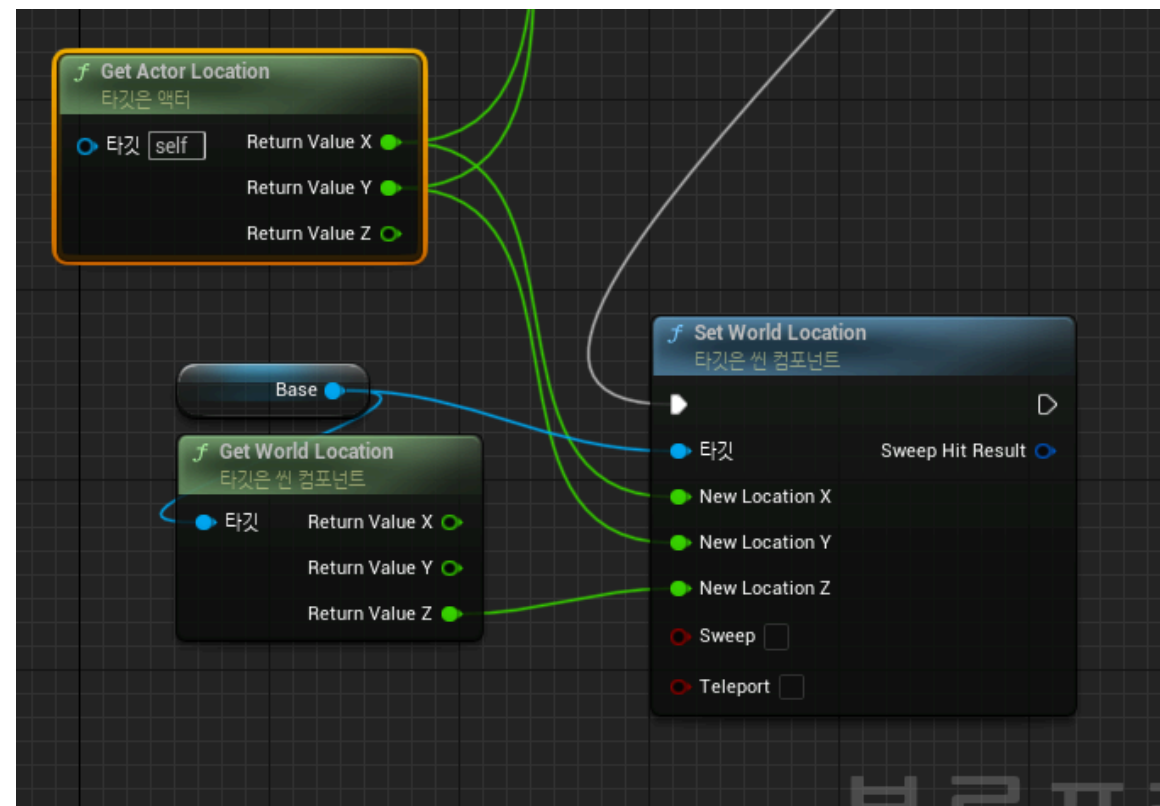
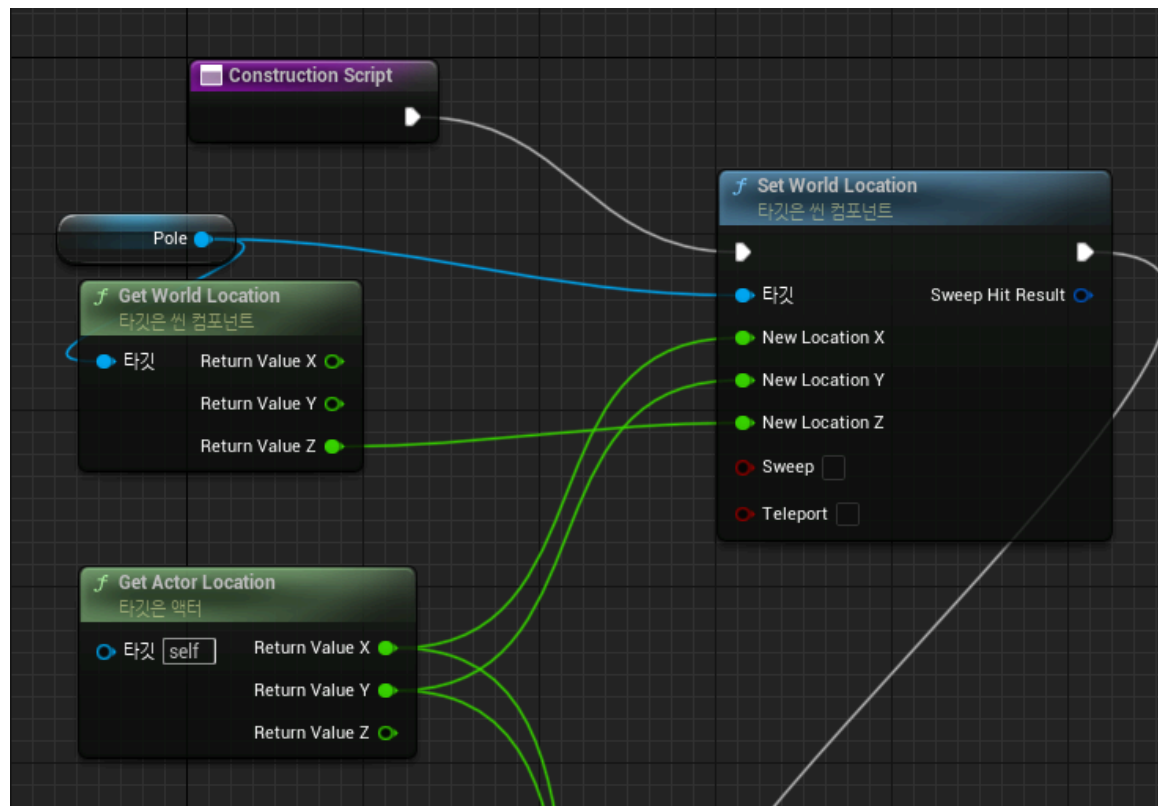
▶ 고급

Pole

- Right now the Pole and Base stay fixed at the center even when the actor moves.
- We will correct their positions to follow the actor:
- set the x, y coordinates of the Pole and Base to the actor's x, y coordinates.

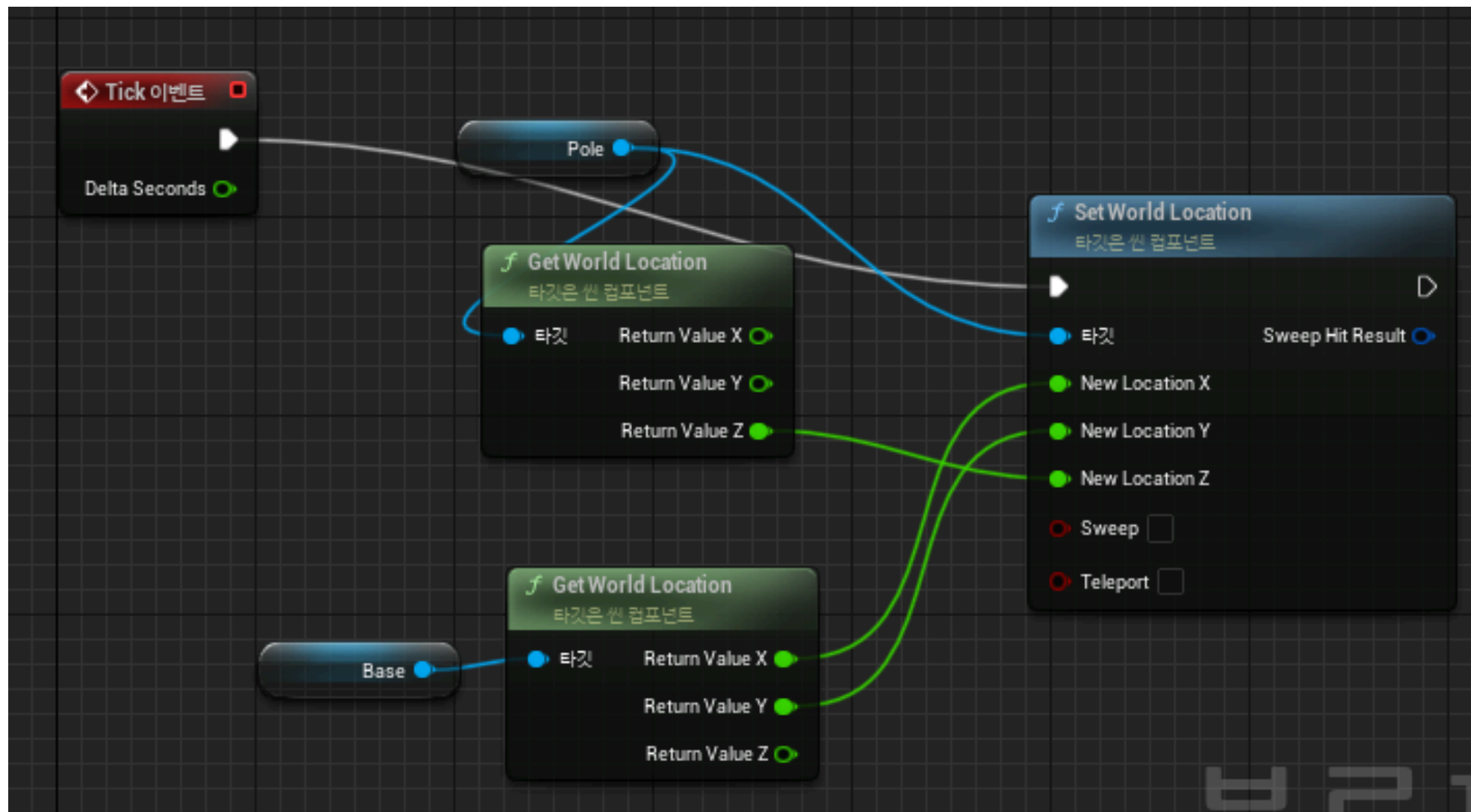


Blueprint - Construction Script



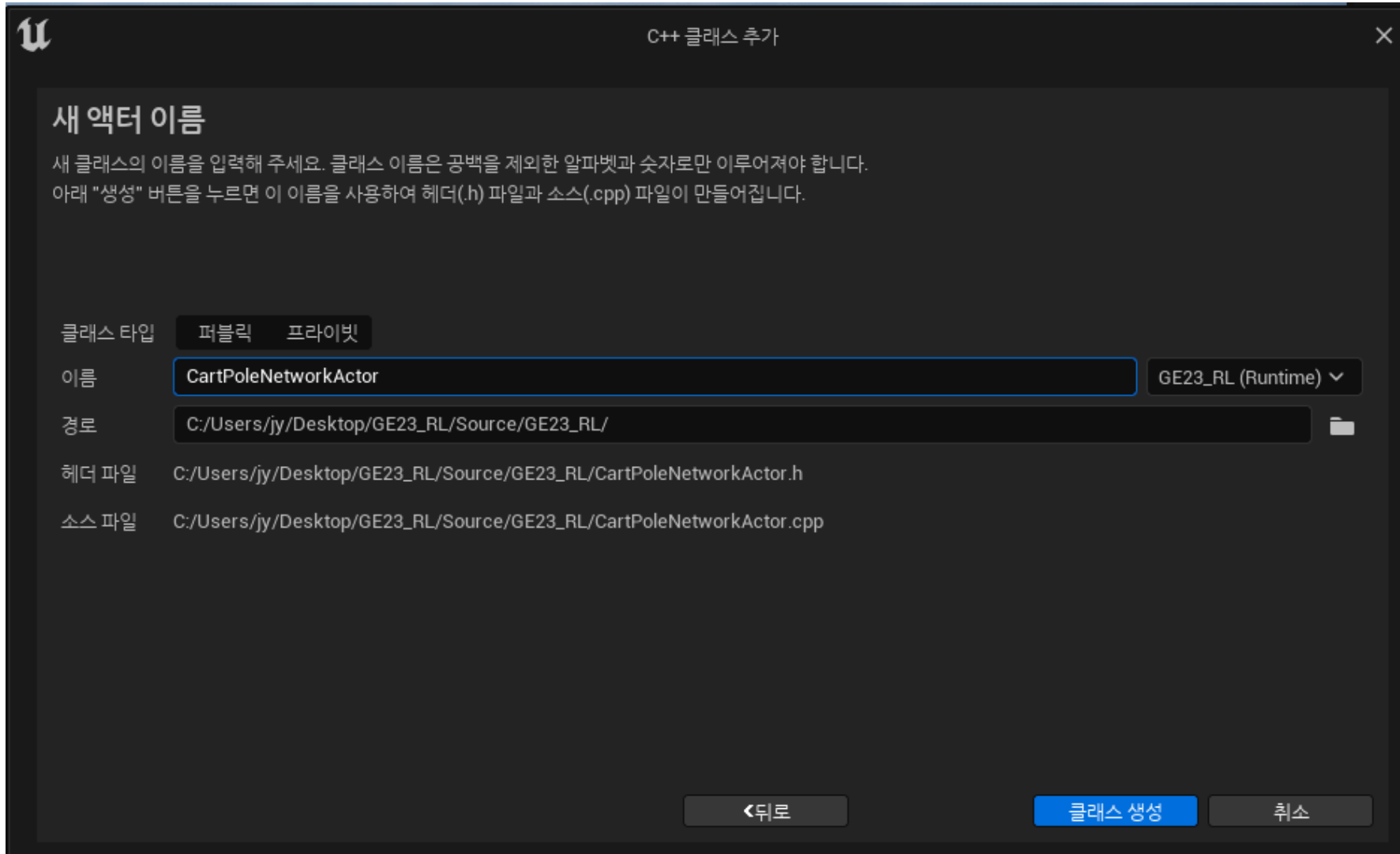
Pole - Event Graph

- Keep the Pole always centered on the Base.



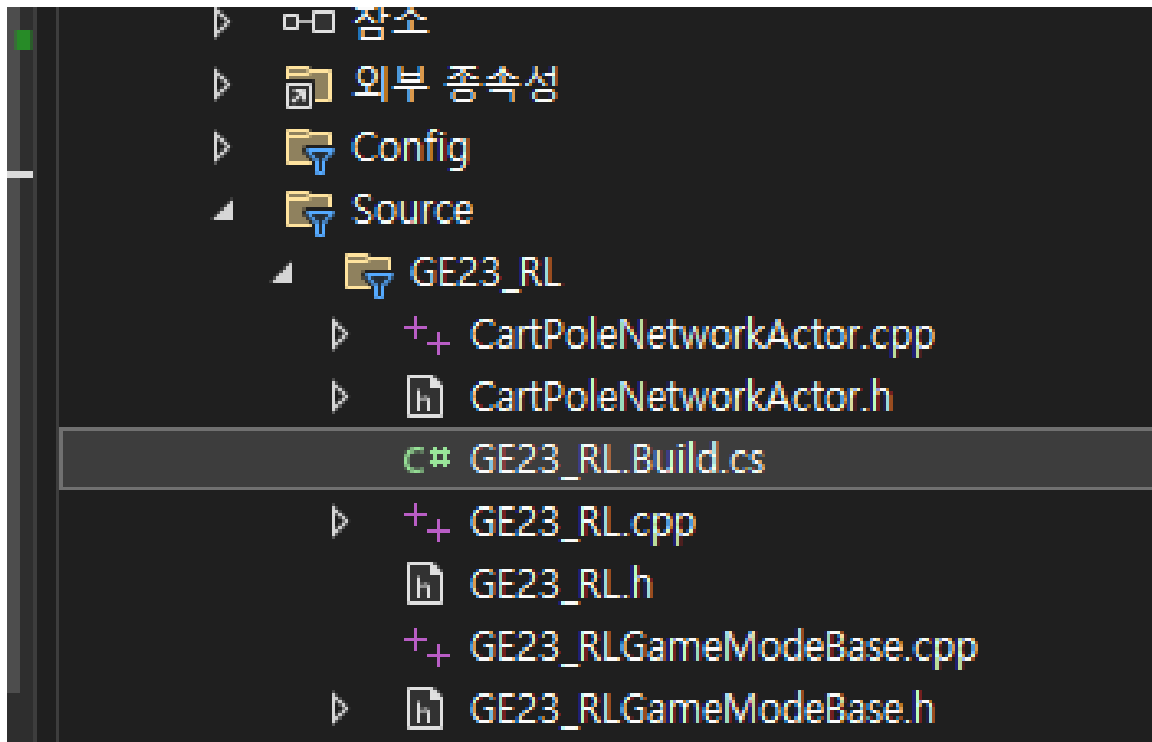
Setup for Networking

- Create a network actor.



GE23_RL.Build.cs

- Add "Networking" and "Sockets" as in the boxes.
- When done, press Ctrl+Alt+F11.



```
using UnrealBuildTool;

public class GE23_RL : ModuleRules
{
    public GE23_RL(ReadOnlyTargetRules Target) : base(Target)
    {
        PCHUsage = PCHUsageMode.UseExplicitOrSharedPCHs;

        PublicDependencyModuleNames.AddRange(new string[] { "Core", "CoreUObject", "Engine", "InputCore", "Networking", "Sockets" });

        PrivateDependencyModuleNames.AddRange(new strings[] { });

        // Uncomment if you are using Slate UI
        // PrivateDependencyModuleNames.AddRange(new string[] { "Slate", "SlateCore" });

        // Uncomment if you are using online features
        // PrivateDependencyModuleNames.Add("OnlineSubsystem");

        // To include OnlineSubsystemSteam, add it to the plugins section in your uproject file with the Enabled attribute set to true
    }
}
```

CartPoleNetworkActor.h

- Add the headers required for networking.
- (Include order matters - follow the screenshot to avoid errors.)
- If Networking.h errors out, use `#include "Runtime/Networking/Public/Networking.h"`.

```
#include "CoreMinimal.h"  
#include "GameFramework/Actor.h"  
#include "Networking.h"  
#include "Async/Async.h"  
#include "Sockets.h"  
#include "CartPoleNetworkActor.generated.h"
```


CartPoleNetworkActor.h

- Create three functions:
- OpenConnection - opens the server.
- CloseConnection - closes the server.
- ManageConnection - manages communication.

CartPoleNetworkActor.h - OpenConnection

```
public:
    // Called every frame
    virtual void Tick(float DeltaTime) override;

    bool IsConnectionOpen = false;
    bool WaitingForConnection = false;
    void OpenConnection();
    void Close_Connection();
    FSocket* ListenSocket = NULL;
```

CartPoleNetworkActor.cpp - OpenConnection

- IPAddress: set the address and port number to connect to.
- ListenSocket indicates that the server is ready to accept clients.

```
void ACartPoleNetworkActor::OpenConnection()
{
    if (!IsConnectionOpen) {
        UE_LOG(LogTemp, Warning, TEXT("[GE23]Opening Connection"));
        IsConnectionOpen = true;
        WaitingForConnection = true;

        FIPv4Address IPAddress;
        FIPv4Address::Parse(FString("127.0.0.1"), IPAddress);
        FIPv4Endpoint Endpoint(IPAddress, (uint16)5050);

        ListenSocket = FTcpSocketBuilder(TEXT("TcpSocket")).AsReusable();

        ISocketSubsystem* SocketSubsystem = ISocketSubsystem::Get(PLATFORM_SOCKETSUBSYSTEM);
        ListenSocket->Bind(*SocketSubsystem->CreateInternetAddr(Endpoint.Address.Value, Endpoint.Port));
        ListenSocket->Listen(1);
        UE_LOG(LogTemp, Warning, TEXT("[GE]Listening"));
    }
}
```

CartPoleNetworkActor.cpp - CloseConnection

- When the connection drops, shut the ListenSocket down as well.

```
void ACartPoleNetworkActor::Close_Connection()
{
    if (IsConnectionOpen) {
        UE_LOG(LogTemp, Warning, TEXT("[GE23]Closing Connection"));
        IsConnectionOpen = false;

        ListenSocket->Close();
    }
}
```

CartPoleNetworkActor h & cpp

- EndPlay signals that the game has ended, just like BeginPlay signals the start.
- Open the server when the game starts and close it when it ends.
- As with BeginPlay, inherit via Super.

```
// Called when the game starts or when spawned
void ACartPoleNetworkActor::BeginPlay()
{
    Super::BeginPlay();
    OpenConnection();
}

void ACartPoleNetworkActor::EndPlay(const EEndPlayReason::Type EndPlayReason)
{
    Super::EndPlay(EndPlayReason);
    Close_Connection();
}
```

```
protected:
    // Called when the game starts or when spawned
    virtual void BeginPlay() override;

    virtual void EndPlay(const EEndPlayReason::Type EndPlayReason) override;
    TFuture<void> ClientConnectionFinishedFuture;
```

CartPoleNetworkActor - ManageConnection

- Receives the data sent from Python.

```
public:
    // Called every frame
    virtual void Tick(Float DeltaTime) override;

    bool IsConnectionOpen = false;
    bool WaitingForConnection = false;
    void OpenConnection();
    void Close_Connection();
    FSocket* ListenSocket = NULL;

    void ManageConnection();
    void ReceiveArrayMessages();
    FSocket* ConnectionSocket = NULL;
    TFuture<void> ClientConnectionFinishedFuture;
```

CartPoleNetworkActor - ManageConnection

- Receives the data sent from Python. Add it to Tick! (code provided)

```

void ACartPoleNetworkActor::ManageConnection()
{
    // Accept Connection
    if (WaitingForConnection) {
        //UE_LOG(LogTemp, Warning, TEXT("Waiting"));
        TSharedRef<FInternetAddr> RemoteAddress = ISocketSubsystem::Get(PLATFORM_SOCKETSUBSYSTEM)->CreateInternetAddr();
        bool hasConnection = false;
        if (ListenSocket->HasPendingConnection(hasConnection) && hasConnection) {
            ConnectionSocket = ListenSocket->Accept(*RemoteAddress, TEXT("[GE23]Connection"));
            WaitingForConnection = false;
            UE_LOG(LogTemp, Warning, TEXT("[GE23]incoming connection"));
            // Start Recv Thread
            ClientConnectionFinishedFuture = Async(EAsyncExecution::LargeThreadPool, [&]() {
                UE_LOG(LogTemp, Warning, TEXT("[GE23]recv thread started"));
                ReceiveArrayMessages();
            });
        }
    }
}

```

CartPoleNetworkActor - ReceiveArrayMessage()

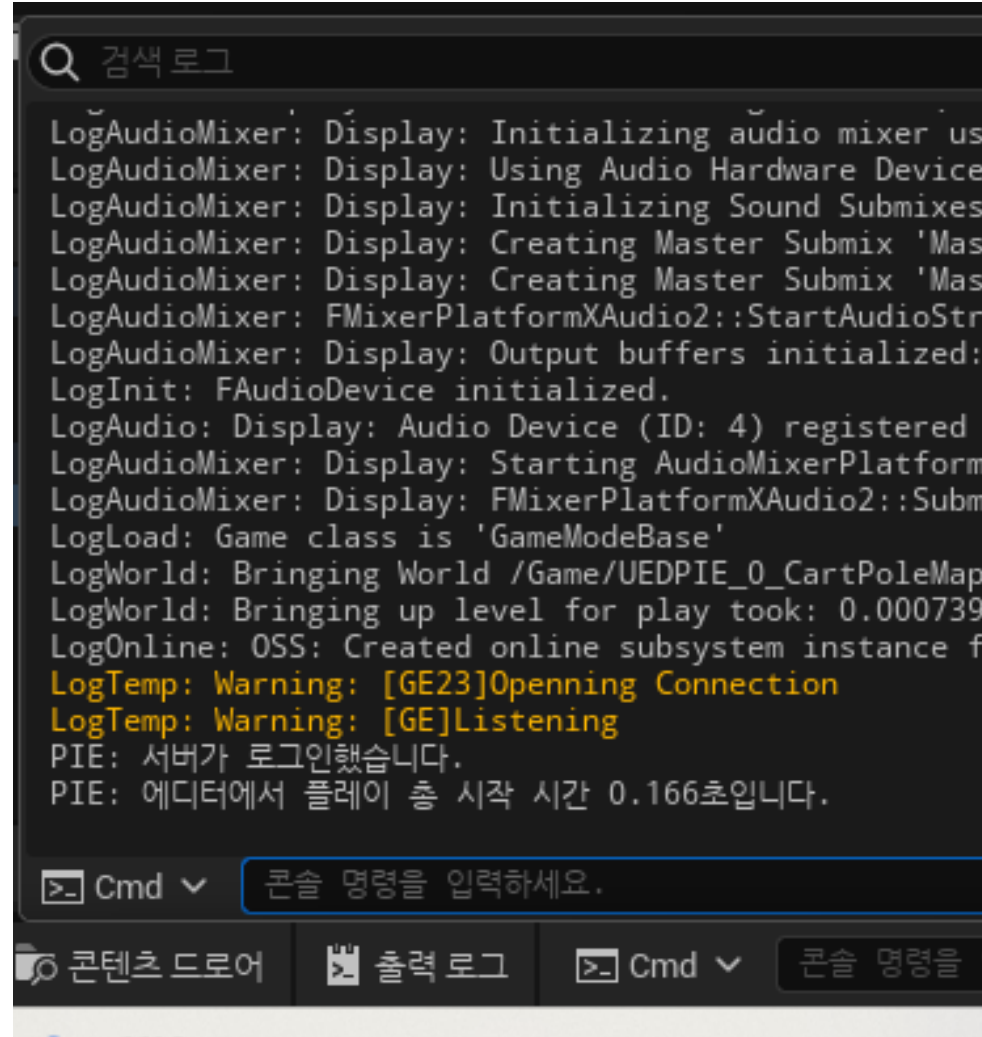
- We will receive an array and process it here (still blank for now).

```
void ACartPoleNetworkActor::ReceiveArrayMessages()
{
    //*****repeat*****
    while (IsConnectionOpen) {
        uint32 size;
        TArray<uint8> ReceivedData;

        if (ConnectionSocket->HasPendingData(size)) {
            ReceivedData.Init(0, 1024);
            int32 Read = 0;
            ConnectionSocket->Recv(ReceivedData.GetData(), ReceivedData.Num(), Read);
            //FString ManageReceivedData = "";
            if (ReceivedData.Num() > 0)
            {
            }
        }
    }
}
```


Test

- 1) Run live coding. 2) Add the network actor to the scene. 3) Play in Unreal.
- 4) Open the Output Log. 5) If it looks like the screenshot, we are ready.



Exchanging Arrays - Python (Client)

- Create clientTest.py to connect to the server.
- Use exactly the same address and port that we set in Unreal.

```
clientTest.py x lab02_gym_cartpole.py x
1  import socket
2  import struct
3  import numpy as np
4  import time
5
6  HOST = '127.0.0.1'
7  PORT = 5050
8
9  client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
10 try:
11     client.connect((HOST, PORT))
12     print("successfully connected")
13 except socket.error:
14     print("couldn't connect", socket.error)
15
```

Exchanging Arrays - Python (Client)

- Create the array. The final array looks like [array size, item1, item2, item3].
- Convert the array data to bytes, then sendall through the client.

```
# 전송할 배열  
array = np.array([1.223, 5.32, 231.23], dtype=np.float32)  
# 배열 크기 추가  
array = np.insert(array, 0, len(array))  
# 배열 크기 및 내용 전송  
array_bytes = array.tobytes()  
client.sendall(array_bytes)
```

Exchanging Arrays - Unreal (That Blank Spot)

- Back in Unreal: read the array size first, then print that many items.

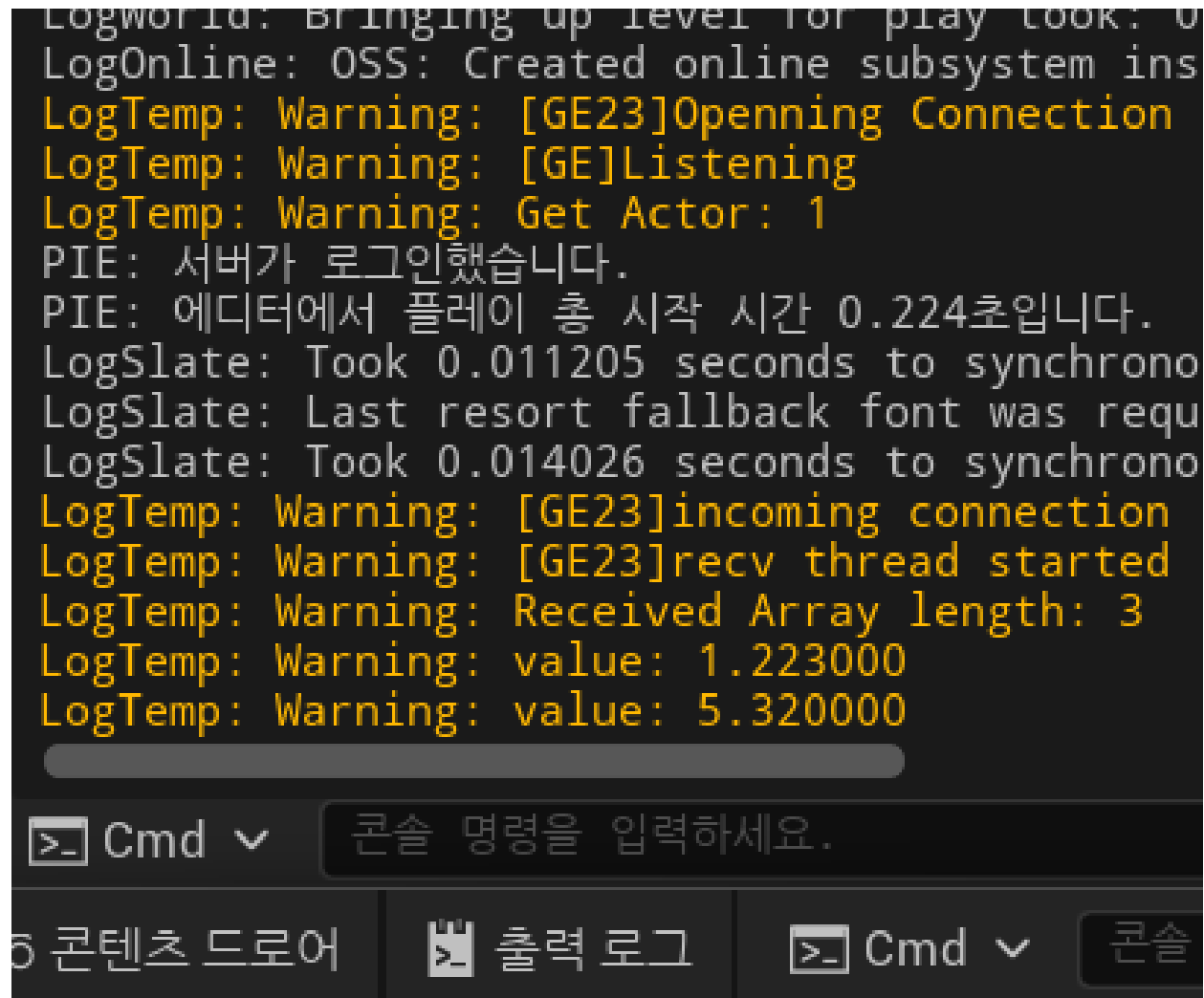
```
int32 Read = 0;
ConnectionSocket->Recv(ReceivedData.GetData(), ReceivedData.Num(), Read);
//FString ManageReceivedData = "";
if (ReceivedData.Num() > 0)
{
    TArray<float> ReceivedArray;
    const int32 numElements = ReceivedData.Num() / sizeof(float);
    //ReceivedArray.SetNumUninitialized(numElements);
    ReceivedArray.SetNum(numElements);
    FMemory::Memcpy(ReceivedArray.GetData(), ReceivedData.GetData(), ReceivedArray.Num());

    int arrayNum = int(ReceivedArray[0]);
    UE_LOG(LogTemp, Warning, TEXT("Received Array length: %d"), arrayNum);
    int id = 0;
    float value = 0;
    for (int i = 1; i < arrayNum; i++) {
        UE_LOG(LogTemp, Warning, TEXT("value: %f"), ReceivedArray[i]);
    }
}
```

Exchanging Arrays

- Run Unreal first, then run Python.
- Python -> Unreal transfer done!

```
LogWorld: Bringing up level for play took: 0
LogOnline: OSS: Created online subsystem ins
LogTemp: Warning: [GE23]Opening Connection
LogTemp: Warning: [GE]Listening
LogTemp: Warning: Get Actor: 1
PIE: 서버가 로그인했습니다.
PIE: 에디터에서 플레이 총 시작 시간 0.224초입니다.
LogSlate: Took 0.011205 seconds to synchrono
LogSlate: Last resort fallback font was requ
LogSlate: Took 0.014026 seconds to synchrono
LogTemp: Warning: [GE23]incoming connection
LogTemp: Warning: [GE23]recv thread started
LogTemp: Warning: Received Array length: 3
LogTemp: Warning: value: 1.223000
LogTemp: Warning: value: 5.320000
```



The screenshot shows the Unreal Engine console interface. The top part displays a series of log messages, including warnings from LogTemp and status messages from LogWorld, LogOnline, LogSlate, and PIE. The bottom part features a command input field with a dropdown menu set to 'Cmd' and a placeholder text '콘솔 명령을 입력하세요.' (Enter console command). Below the command field, there are three buttons: '콘텐츠 드로어' (Content Drawer), '출력 로그' (Output Log), and another 'Cmd' dropdown with '콘솔' (Console) selected.

Exchanging Arrays - Unreal (Server)

- Below where we just received the array, add the array send code.
- It sends the size of the FloatArray, then the data.

```

//// ===== Array Receive =====
int arrayNum = int(ReceivedArray[0]);
UE_LOG(LogTemp, Warning, TEXT("Received Array length: %d"), arrayNum);
int id = 0;
float value = 0;
for (int i = 1; i < arrayNum; i++) {
    UE_LOG(LogTemp, Warning, TEXT("value: %f"), ReceivedArray[i]);
}

// ===== Array Send =====

//보낼 배열
TArray<float> FloatArray = { 1.2434f, 3.12432f, 2.34234f, 10.01231f};

uint32 ArraySize = FloatArray.Num();
int32 sent = 0;
ConnectionSocket->Send(reinterpret_cast<const uint8*>(&ArraySize), sizeof(uint32), sent);

TArray<uint8> ArrayData;
ArrayData.SetNumUninitialized(ArraySize * sizeof(float));
FMemory::Memcpy(ArrayData.GetData(), FloatArray.GetData(), ArrayData.Num());
sent = 0;
ConnectionSocket->Send(ArrayData.GetData(), ArrayData.Num(), sent);

// ===== Array End =====

```

Exchanging Arrays - Python (Client)

- Back in Python: convert the array size to an integer and read the array data.

```
# 배열 크기 수신
start = time.time()
array_size_bytes = client.recv(4)
array_size = struct.unpack('!I', array_size_bytes)[0]
#4바이트 정수 데이터를 해석하여 원래의 정수 값으로 언패킹하는 역할
# 배열 데이터 수신
array_data_bytes = client.recv(array_size * 4)
array_data = np.frombuffer(array_data_bytes, dtype=np.float32)
end = time.time()
# 수신한 배열 출력
print("Received Array:", array_data)
print(f"total time: {end-start} ms")
# 연결 종료
client.close()
```

Exchanging Arrays

- Below is the Python log window.
- Unreal -> Python transfer done!

```
C:\Users\user\anaconda3\python.exe C:\Users\user\Desktop\GE  
successfully connected  
Received Array: [ 1.2434    3.12432  2.34234 10.01231]  
total time: 0.007001399993896484 ms  
  
Process finished with exit code 0
```


Unreal-Python Communication

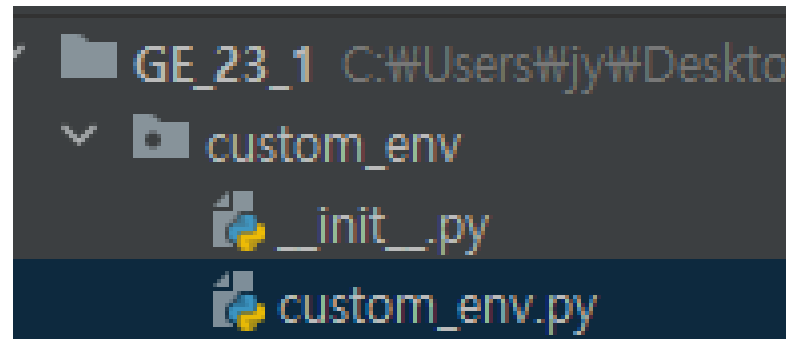
- The communication groundwork is done. RL repeats Reset and Step to collect data:
- 1) Calling `reset()` in Python tells Unreal to reset its state and returns that state.
- 2) Python selects an action based on that state.
- 3) Calling `step(action)` tells Unreal to move with that action and returns the new state.
- Repeat until the episode ends.

Unreal-Python Communication - Flow Recap

- 1. Run Unreal -> server is ready.
- 2. Run Python -> `env = custom_env()` connects the client.
- 3. Python `env.reset()` -> sends an array/number -> Unreal receives it and sends the new state (array) -> Python receives it in a while loop.
- 4. Python `env.step(env.action_space.sample())` -> sends an array/number -> Unreal receives it and sends the new state -> Python receives it in a while loop.

Unreal-Python Communication

- In the Python folder: top folder > right-click > New > Python Package > create "custom env".
- Create custom_env.py inside as well.



Unreal-Python Communication

- Connect the socket as soon as the class is created.

```
class custom_env(gym.Env):  
    def __init__(self):  
        self.action_space = gym.spaces.Discrete(2)  
        self.observation_space = gym.spaces.Box(-180, 180, [1])  
  
        print(self.observation_space.high)  
        print(self.observation_space.low)  
  
        self.hasConnection = False  
        self.socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)  
  
        try:  
            self.socket.connect((HOST, PORT))  
            print("successfully connected")  
            self.hasConnection = True  
        except socket.error:  
            print("couldn't connect", socket.error)
```

Unreal-Python Communication - step()

- step() sends the action and receives the new state.
- For now we only send [action].

```
def step(self, action):  
  
    #전송할 배열  
    array = np.array([action], dtype=np.float32)  
    # 배열 크기 추가  
    #array = np.insert(array, 0, len(array))  
    # 배열 크기 및 내용 전송  
    array_bytes = array.tobytes()  
    self.socket.sendall(array_bytes)  
  
    state = []
```

Unreal-Python Communication - step()

- Wait inside a while loop until the new state arrives from Unreal.
- When Unreal sends (pole angle, cart position, cart velocity, pole velocity), copy it into state.

```
# wait till you get a response
while (True):
    array_size_bytes = self.socket.recv(4)
    if not array_size_bytes:
        break
    array_size = struct.unpack('!I', array_size_bytes)[0] # 4바이트 정수 데이터를
    # 배열 데이터 수신
    array_data_bytes = self.socket.recv(array_size * 4)
    array_data = np.frombuffer(array_data_bytes, dtype=np.float32)
    # print("Step Array:", array_data)
    state = array_data.copy()
    break
```

Unreal-Python Communication - step()

- State = (pole angle, cart position, cart velocity, pole velocity).
- Terminate when the angle exceeds 30 degrees or the cart strays too far from the center.

```
reward = 1

done = False
if (abs(state[0]) > 30):
    done = True
elif (abs(state[1] > 800)):
    done = True

return state, reward, done
```

Unreal-Python Communication

- When Unreal receives 10, it resets the environment.
- Use a zero-initialized state instead of the received one.

```
def reset(self):  
  
    array = np.array([10], dtype=np.float32)  
    # array = np.insert(array, 0, len(array))  
    array_bytes = array.tobytes()  
    self.socket.sendall(array_bytes)  
  
    state = [1, 1, 1, 1]  
  
    # wait till you get a response  
    while (True):  
        array_size_bytes = self.socket.recv(4)  
        if not array_size_bytes:  
            break  
        array_size = struct.unpack('!I', array_size_bytes)[0]  
        # 배열 데이터 수신  
        array_data_bytes = self.socket.recv(array_size * 4)  
        array_data = np.frombuffer(array_data_bytes, dtype=np.float32)  
        # print("Reset Array:", array_data)  
        break  
  
    state = [0, 0, 0, 0]  
    return state
```


Unreal-Python Communication - PoleActor

- We need functions that reset the CartPole on 10, and move it on 0 or 1.

```
void Reset_myEnv(int id);  
void Move_XAxis(int Input_);  
void ApplySpeed(float d);
```

```
UPROPERTY(EditAnywhere)  
- UE 청사진용 CodeLens  
float MaxSpeed;
```

```
UPROPERTY(EditAnywhere)  
- UE 청사진용 CodeLens  
float MotorPower;
```

```
UPROPERTY(EditAnywhere)  
- UE 청사진용 CodeLens  
float CurrMotorSpeed;
```

Unreal-Python Communication - Reset

- The cart moves along the X axis, so multiple PoleActors are lined up along the Y axis (300 * id).
- Also fix the Pole's angle and position, and finally reset the Pole's vector.

```
void APoleActor::Reset_myEnv(int id)
{
    FVector currLocation = Base->GetComponentLocation();
    currLocation.X = 0.0f;
    currLocation.Y = 300.0f * id;
    currLocation.Z = 0.0f;
    Base->SetWorldLocation(currLocation);

    CurrMotorSpeed = 0;

    FRotator rotator = Pole->GetRelativeRotation();
    rotator.Pitch = 0.0f;
    rotator.Roll = 0.0f;
    rotator.Yaw = 0.0f;
    Pole->SetRelativeRotation(rotator);

    currLocation = Pole->GetComponentLocation();
    currLocation.X = 0.0f;
    currLocation.Y = 300.0f * id;
    currLocation.Z = 10.0f;
    Pole->SetWorldLocation(currLocation);

    FVector angularVel = FVector();
    Pole->SetPhysicsAngularVelocityInDegrees(angularVel);
    Pole->SetAllPhysicsLinearVelocity(angularVel);
}
```

Unreal-Python Communication - MoveXAxis

- MoveXAxis adjusts Speed according to the received action (AxisValue).
- Set the Base position from that Speed, then damp the speed slightly by multiplying by 0.9.

```
void APoleActor::Move_XAxis(int AxisValue)
{
    if (AxisValue == 0) AxisValue = -1;
    UE_LOG(LogTemp, Warning, TEXT("Move %d"), AxisValue);
    CurrMotorSpeed += AxisValue * MotorPower;
    CurrMotorSpeed = FMath::Clamp(CurrMotorSpeed, -MaxSpeed, MaxSpeed);
}

void APoleActor::ApplySpeed(float DeltaTime)
{
    FVector currLocation = Base->GetComponentLocation();
    currLocation.X = currLocation.X + (CurrMotorSpeed * DeltaTime);
    Base->SetWorldLocation(currLocation);
    CurrMotorSpeed *= 0.9f;
}
```

Unreal-Python Communication - Network

- CartpoleNetwork finds every PoleActor in the scene, then drives them by the values coming from Python.
- e.g. (1,10) resets PoleActor #1; (0,0) moves PoleActor #0 to the left; (10,1) moves PoleActor #10 to the right.

CartPoleNetwork.h

- "Kismet/GameplayStatics.h" is needed to find actors in the scene.
- (Include order matters - follow the screenshot to avoid errors.)

```
#include "CoreMinimal.h"
#include "GameFramework/Actor.h"
#include "Networking.h"
#include "Async/Async.h"
#include "Sockets.h"
#include "Kismet/GameplayStatics.h"
#include "PoleActor.h"
#include "CartPoleNetworkActor.generated.h"
```

```
FSocket* ConnectionSocket = NULL;

TArray<APoleActor*> poles;
```

```
protected:
    // Called when the game starts or when spawned
    virtual void BeginPlay() override;

    virtual void EndPlay(const FEndPlayEvent& Event) override;

    bool HasInput;
    bool AppliedInput;
    int Input;
```

CartPoleNetwork.cpp

- Find all PoleActors.
- Variables: HasInput - whether there is input; AppliedInput - whether it was applied; Input - the received value.

```

// Called when the game starts or when spawned
void ACartPoleNetworkActor::BeginPlay()
{
    Super::BeginPlay();
    OpenConnection();

    TSubclassOf<APoleActor> classToFind;
    TArray<AActor*> find;
    classToFind = APoleActor::StaticClass();
    UGameplayStatics::GetAllActorsOfClass(GetWorld(), classToFind, find);
    UE_LOG(LogTemp, Warning, TEXT("Get Actor: %d"), find.Num());

    //actor 타입에서 PoleActor 타입으로 변경해줌
    APoleActor* caster = NULL;

    for (int i = 0; i < find.Num(); i++) {
        caster = Cast<APoleActor>(find[i]);
        // caster->Reset_myEnv(i);
        poles.Add(caster);
    }

    HasInput = false;
    AppliedInput = false;
    Input = 0;
}

```

Unreal-Python - Tick Flow

- Tick proceeds in this flow. Two things to change:
- ManageConnection (modify) and SendObservation (add).

```
// Called every frame
void ACartPoleNetworkActor::Tick(float DeltaTime)
{
    Super::Tick(DeltaTime);

    ManageConnection(); //python으로부터 값 받아옴

    if (Input == 10) { //reset 차례
        poles[0]->Reset_myEnv(0);
    }
    else if (Input == 11) { //아래서 sendobservation할거임
    }
    else { //reset도 아니고, reset 후 state를 보내야하는 차례도 아니라면
        poles[0]->ApplySpeed(DeltaTime);
    }

    //=====
    if (Input != 10 && AppliedInput) {
        AppliedInput = false;
        OnPause();
        SendObservation();
    }

    if (Input == 10) { // sendobservation 준비
        Input = 11;
    }

    if (HasInput) { // 다음 Input을 위한 준비
        HasInput = false;
    }
}
```

Unreal-Python - Tick()

- ManageConnection() -> receive whether to reset or move.
- If (Reset) -> Pole.ResetEnv(); else -> Pole.ApplySpeed().
- If (reset finished or moved) -> send the state.
- If (HasInput) -> set HasInput = false for the next input.
- If (Reset) -> reset is done and ready to send -> change 10 to 11.

ManageConnection - ReceiveArrayMessage

- Comment out the array-exchange demo code that was in the blank spot earlier.

```
if (ReceivedData.Num() > 0)
{
    TArray<float> ReceivedArray;
    const int32 numElements = ReceivedData.Num() / sizeof(float);
    //ReceivedArray.SetNumUninitialized(numElements);
    ReceivedArray.SetNum(numElements);
    FMemory::Memcpy(ReceivedArray.GetData(), ReceivedData.GetData(), ReceivedArray.Num());

    Input = int(ReceivedArray[0]);

    UE_LOG(LogTemp, Warning, TEXT("Value: %d"), Input);
    HasInput = true;

    // Handle Input
    if (HasInput) {
        OnResume();
        AppliedInput = true;
        if (Input == 10) {
            // nothing!
        }
        else {
            poles[0] -> Move_XAxis(Input);
        }
    }
}
```

SendObservation h & cpp

- Add it to the header, then add the body below.

```
void ACartPoleNetworkActor::SendObservation()
{
    FVector vel2 = poles[0]->Pole->GetPhysicsAngularVelocityInDegrees();
    FVector vel = poles[0]->Base->GetPhysicsLinearVelocity();
    FRotator rotator = poles[0]->Pole->GetRelativeRotation();
    FVector currLocation = poles[0]->Base->GetComponentLocation();
    //보낼 배열
    TArray<float> NewState = { float(rotator.Pitch), float(currLocation.X), float(vel.X), float(vel2.Y) };

    uint32 ArraySize = NewState.Num();
    int32 sent = 0;
    ConnectionSocket->Send(reinterpret_cast<const uint8*>(&ArraySize), sizeof(uint32), sent);

    TArray<uint8> ArrayData;
    ArrayData.SetNumUninitialized(ArraySize * sizeof(float));
    FMemory::Memcpy(ArrayData.GetData(), NewState.GetData(), ArrayData.Num());
    sent = 0;
    ConnectionSocket->Send(ArrayData.GetData(), ArrayData.Num(), sent);
}
```

Python - custom_env.py

- Now prepare Python. Add the code below at the very bottom of custom_env.py.
- The structure is identical to Gym - only the rendering happens in Unreal.

```
env = custom_env()

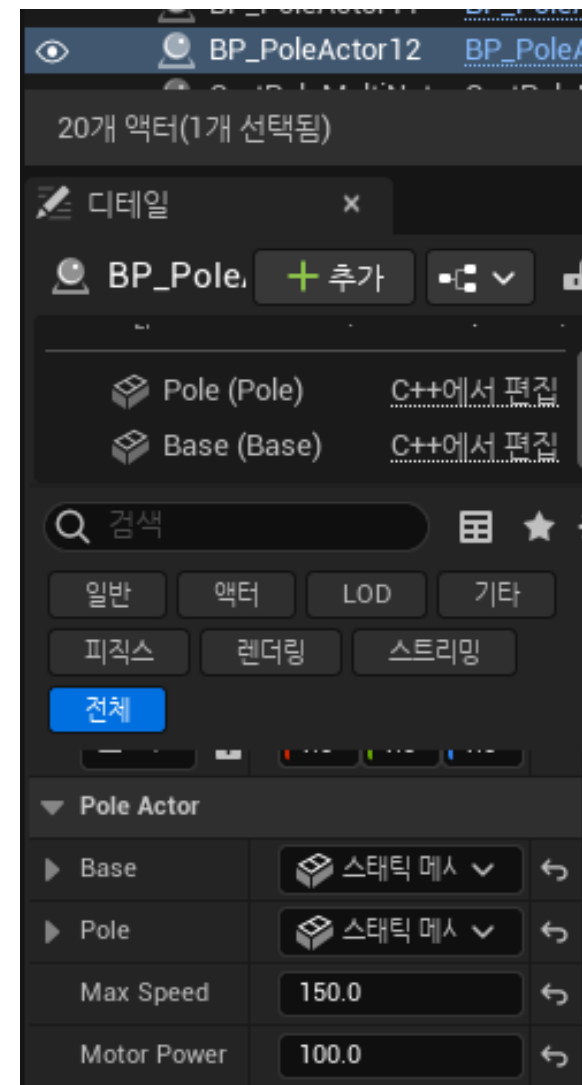
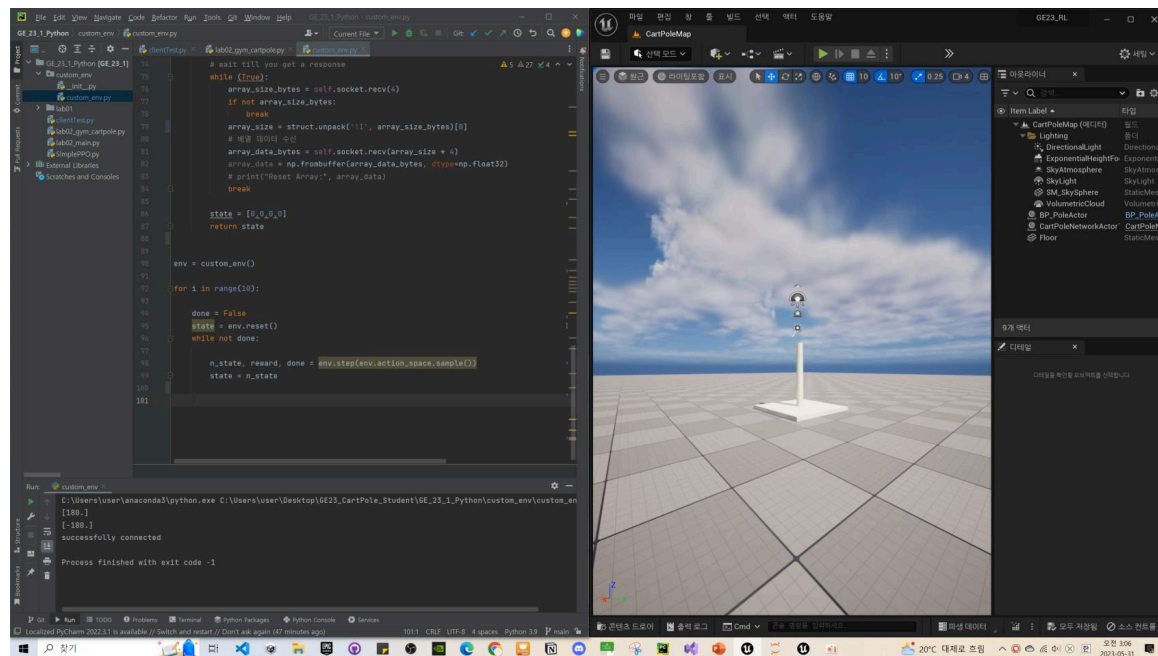
for i in range(10):

    done = False
    state = env.reset()
    while not done:

        n_state, reward, done = env.step(env.action_space.sample())
        state = n_state
```

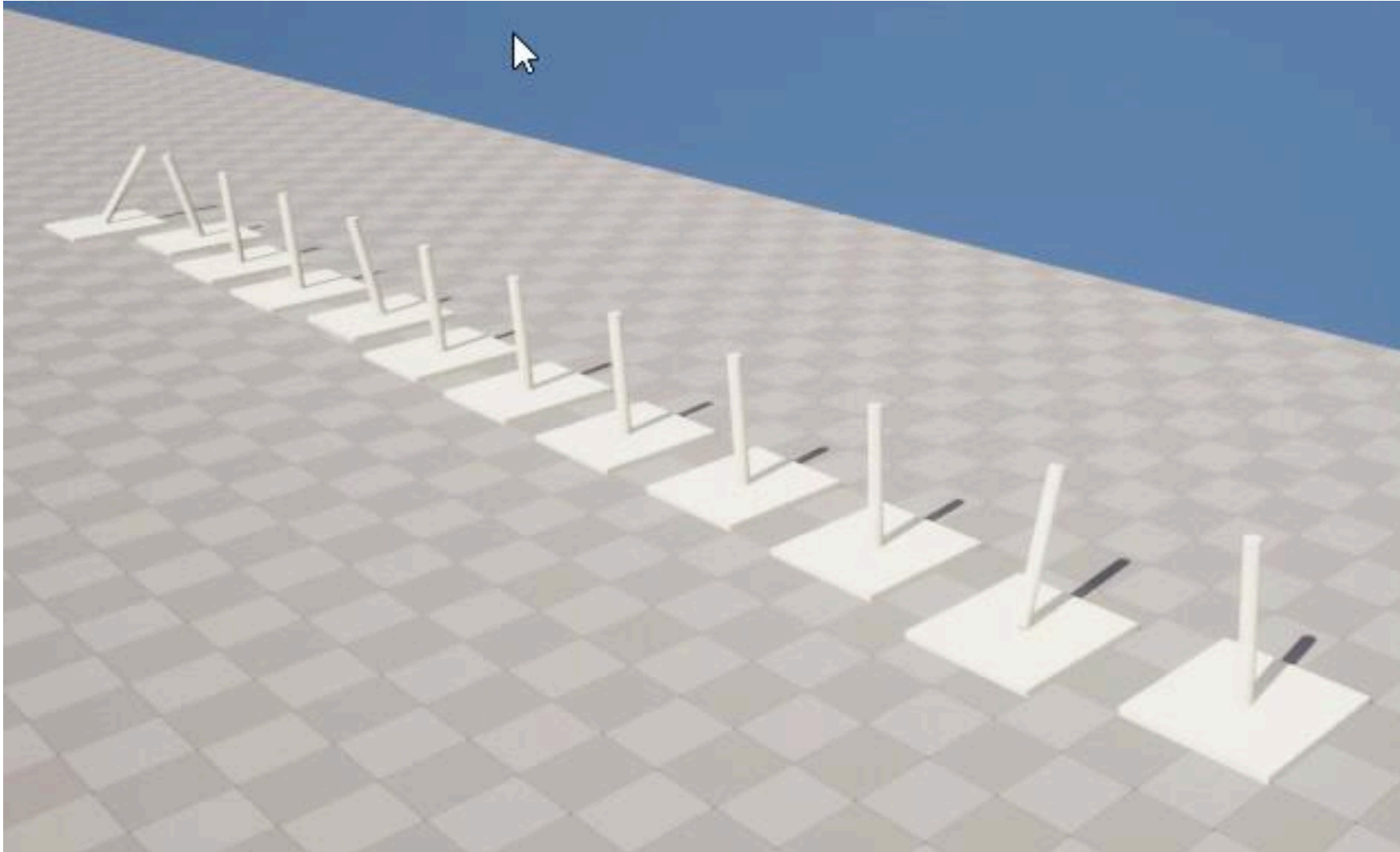
Single Env Complete

- Finally, click the PoleActor in the Outliner and edit it as in the red box.



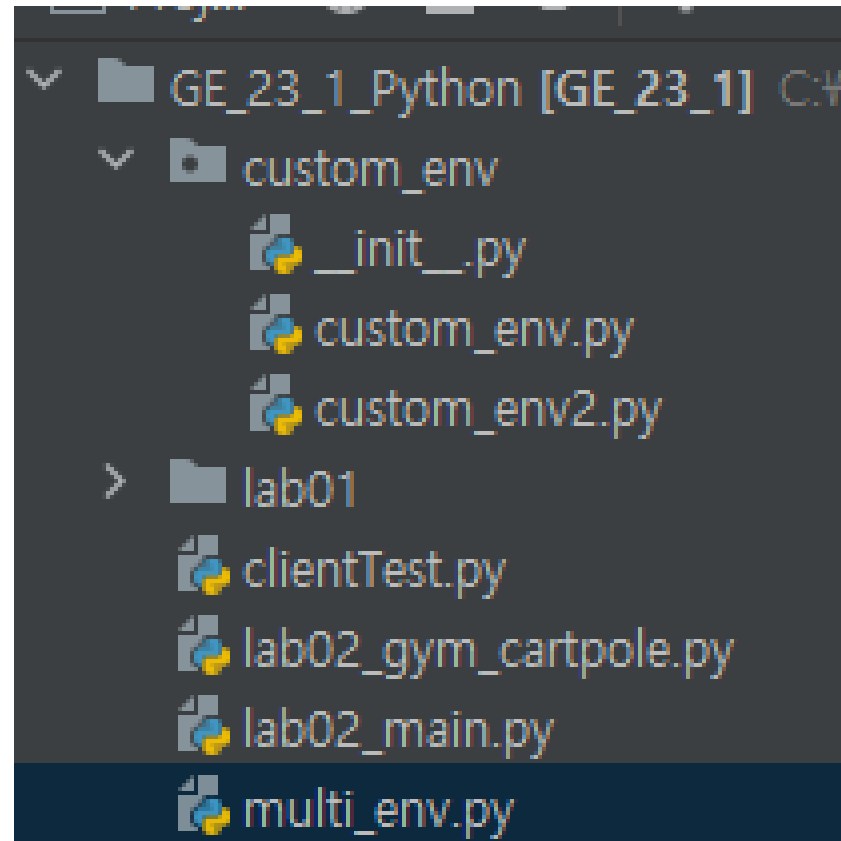
Multi-Env

- Before training, we switch from a single env to multiple envs - mainly for training speed.



Multi-Env

- Send the env id together with the value: [value] becomes [id, value].
- Add multi_env.py to create multiple environments.
- custom_env changes too - either create custom_env2.py or edit the existing code, whichever is convenient.
- Python Env 01..N <-> Unreal PoleActor 01..N



Multi-Env

- Move the connection into multi-env.
- Similar to custom_env, but now it creates num_env environments.

```
from custom_env import custom_env2
import socket

HOST = '127.0.0.1'
PORT = 5050

# 황주영
class multi_env:
    # 황주영
    def __init__(self, num_env):
        self.envs = []

        for _ in range(num_env):
            env = custom_env2.custom_env(_)
            self.envs.append(env)

        self.hasConnection = False
        self.socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

        try:
            self.socket.connect((HOST, PORT))
            print("successfully connected")
            self.hasConnection = True
        except socket.error:
            print("couldn't connect", socket.error)
```

Multi-Env

- Likewise, create the reset and step functions - reset initializes per id.

황주영 *

```
def reset(self, id):  
    state = []  
    return state
```

황주영 *

```
def step(self, actions):  
    obs = []  
    rewards = []  
    dones = []  
  
    return obs, rewards, dones
```


Custom-Env

- Add id to `__init__` and remove the connection part (delete everything below the print in the screenshot).
- Add `self.id` to the arrays sent in step and reset.
- The socket moved to multi-env, so receive it as client - replace every `self.socket` with `client`.

```
def step(self, client, action):
    #전송할 배열
    array = np.array([self.id, action], dtype=np.float32)
```

```
def reset(self, client):
    array = np.array([self.id, 10], dtype=np.float32)
```

```
class custom_env(gym.Env):
    # 환경
    def __init__(self, id):
        self.id = id #multi env에서 생성될 때 할당
        self.action_space = gym.spaces.Discrete(2)
        self.observation_space = gym.spaces.Box(-180, 180, [1])

        print(self.observation_space.high)
        print(self.observation_space.low)
```

Multi-Env

- Now fill in the multi-env functions.
- `step()` receives an action array with one entry per env - e.g. `[0,1,1]` for 3 envs.
- Because it owns the socket, reset is called inside when done.
- The next state `ob` is a `numpy.ndarray`, so call `.tolist()` to convert it to a list.

```
def reset(self, id):
    state = self.envs[id].reset(self.socket)
    return state
```

활주영 +1

```
def step(self, actions):
    obs = []
    rewards = []
    done = []

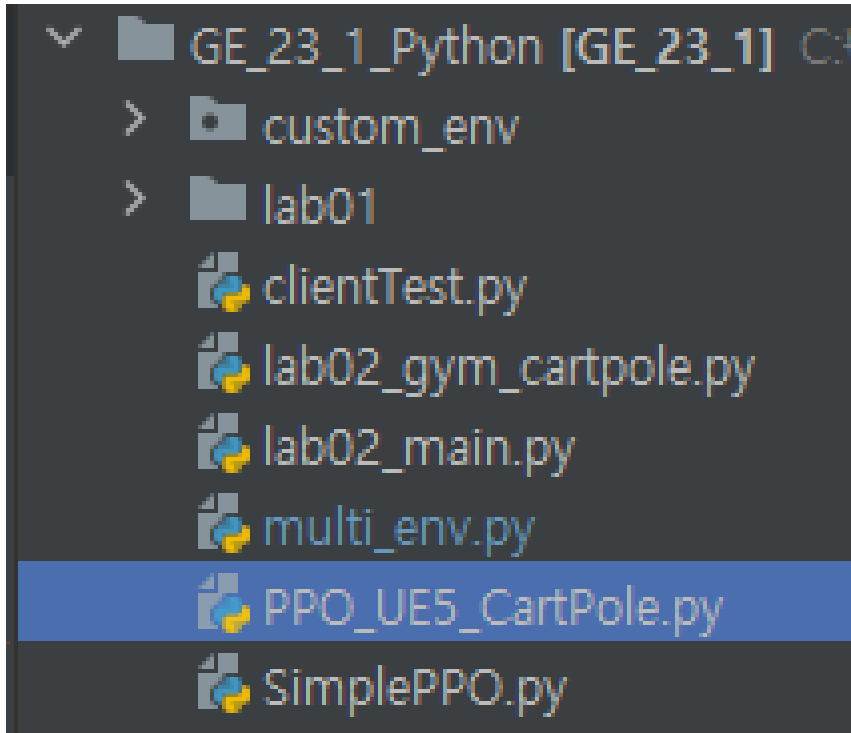
    for i in range(len(actions)):
        env = self.envs[i]
        ac = actions[i]
        ob, rew, done = env.step(self.socket, ac)
        obs.append(ob.tolist())
        rewards.append(rew)
        done.append(done)

        if done:
            env.reset(self.socket)

    return obs, rewards, done
```

PPO_UE5_CartPole.py

- Create one file that orchestrates the multi-env.
- Create envcount envs, reset them all, generate random actions and send them to step.
- Python side is ready!



```
from multi_env import multi_env

ENVCOUNT = 3
multi_env = multi_env(ENVCOUNT)

for id in range(ENVCOUNT):
    multi_env.reset(id)

for timestep in range(1000):
    actions = []
    for s in range(ENVCOUNT):
        actions.append(multi_env.envs[s].action_space.sample())
    print(actions)
    n_states, rewards, dones = multi_env.step(actions)
```

network.h

- HasInput, AppliedInput and Input are now needed per env, so turn them into arrays.
- Add an int id parameter to send the information of poles[id].

```
public:
    // Sets default values for this actor's properties
    ACartPoleMultiNetwork();
    virtual void EndPlay(const EEndPlayReason::Type EndPlayReason) override;

    //*****new*****
    TArray<bool> HasInput;
    TArray<bool> AppliedInput;
    TArray<int> Input;
```

```
//*****new*****
void SendObservation(int id);
```

network.cpp

- Modify the existing code in BeginPlay.
- Wrap the Tick contents in a for loop and add the id.

```

37     // **** new ****
38     HasInput.Init(false, poles.Num());
39     AppliedInput.Init(false, poles.Num());
40     Input.Init(0, poles.Num());
41 }
42

```

```

// Called every frame
void ACartPoleMultiNetwork::Tick(float DeltaTime)
{
    Super::Tick(DeltaTime);

    // ***** new *****

    for (int id = 0; id < poles.Num(); id++) {

        ManageConnection(); //python으로부터 값 받아옴

        if (Input[id] == 10) { //reset 차례
            poles[id] -> Reset_myEnv(id);
        }
        else if (Input[id] == 11) { //아래서 sendobservation할거임
        }
        else { //reset도 아니고, reset 후 state를 보내야하는 차례도 아니라면
            poles[id] -> ApplySpeed(DeltaTime);
        }

        //=====
        if (Input[id] != 10 && AppliedInput[id]) {
            AppliedInput[id] = false;
            OnPause();
            // ***** new *****U
            SendObservation(id);
            // ***** new *****U
        }

        if (Input[id] == 10) { // sendobservation 준비
            Input[id] = 11;
        }

        if (HasInput[id]) { // 다음 Input을 위한 준비
            HasInput[id] = false;
        }
    }
}

```

network.cpp

```

ReceivedData.Init(0, 1024);
int32 Read = 0;
ConnectionSocket->Recv(ReceivedData.GetData(), ReceivedData.Num(), Read);
// FString ManageReceivedData = "";
if (ReceivedData.Num() > 0)
{
    TArray<float> ReceivedArray;
    const int32 numElements = ReceivedData.Num() / sizeof(float);
    // ReceivedArray.SetNumUninitialized(numElements);
    ReceivedArray.SetNum(numElements);
    FMemory::Memcpy(ReceivedArray.GetData(), ReceivedData.GetData(), ReceivedData.Num());

    // ***** new *****U
    int _id = int(ReceivedArray[0]);
    Input[_id] = int(ReceivedArray[1]);

    UE_LOG(LogTemp, Warning, TEXT("Env id: %d, Value: %d"), _id, Input[_id]);
    HasInput[_id] = true;

    // Handle Input
    if (HasInput[_id]) {
        OnResume();
        AppliedInput[_id] = true;
        if (Input[_id] == 10) {
            // nothing!
        }
        else {
            poles[_id]->Move_XAxis(Input[_id]);
        }
    }

    // ***** new *****U
}

```

```

void ACartPoleMultiNetwork::SendObservation(int id)
{
    // ***** new *****U
    FVector vel2 = poles[id]->Pole->GetPhysicsAngularVelocityInDegrees();
    FVector vel = poles[id]->Base->GetPhysicsLinearVelocity();
    FRotator rotator = poles[id]->Pole->GetRelativeRotation();
    FVector currLocation = poles[id]->Base->GetComponentLocation();
    // ***** new *****U
    //보낼 배열
    TArray<float> NewState = { float(rotator.Pitch), float(currLocation.X), float(vel.X), float(vel2.Y) };
    UE_LOG(LogTemp, Warning, TEXT("State: %f, %f, %f, %f"), NewState[0], NewState[1], NewState[2], NewState[3]);

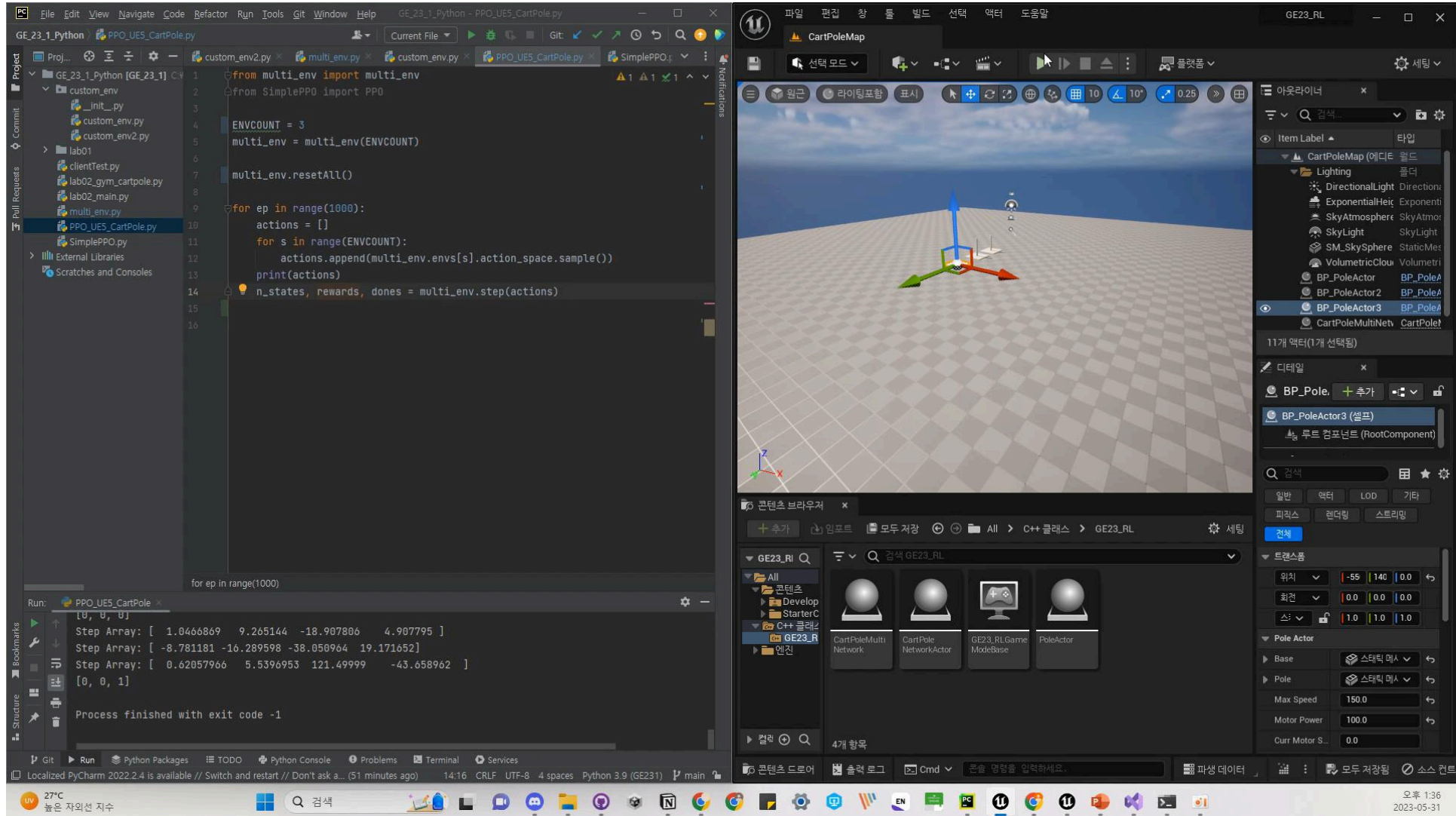
    uint32 ArraySize = NewState.Num();
    int32 sent = 0;
    ConnectionSocket->Send(reinterpret_cast<const uint8*>(&ArraySize), sizeof(uint32), sent);

    TArray<uint8> ArrayData;
    ArrayData.SetNumUninitialized(ArraySize * sizeof(float));
    FMemory::Memcpy(ArrayData.GetData(), NewState.GetData(), ArrayData.Num());
    sent = 0;
    ConnectionSocket->Send(ArrayData.GetData(), ArrayData.Num(), sent);
}

```

Final Result

- As many PoleActors are spawned as the envcount set in Python.



Multi Env + PPO

- Back in Python, add the agent.
- Create arrays to track each env's current episode reward and the reward received per episode.
- Use the agent to select actions, and store the probability and the chosen action.

```
from multi_env import multi_env
from SimplePPO import PPO

ENVCOUNT = 3
multi_env = multi_env(ENVCOUNT)
agent = PPO(input_dim=4, action_dim=2)

episode_history = [[] for _ in range(ENVCOUNT)]
episode_rewards = [0 for x in range(ENVCOUNT)]
print(episode_history)
print(episode_rewards)

states = []
for id in range(ENVCOUNT):
    states = multi_env.reset(id)

for timestep in range(1000):
    actions = []
    probs = []

    for id in range(ENVCOUNT):
        prob, action = agent.select_action(states[id])
        actions.append(action)
        probs.append(prob[action].item())

    n_states, rewards, dones = multi_env.step(actions)
```


Multi Env + PPO

```
# print(actions)
n_states, rewards, dones = multi_env.step(actions)

for i in range(ENVCOUNT):
    transition = (states[i], actions[i], rewards[i], n_states[i], probs[i], dones[i])
    # print(transition)
    agent.put_data((states[i], actions[i], rewards[i], n_states[i], probs[i], dones[i]))
    episode_rewards[i] += rewards[i]

    if(dones[i]):
        multi_env.reset(i)
        episode_history[i].append(episode_rewards[i])
        episode_rewards[i] = 0

probs.clear()
actions.clear()
states.clear()

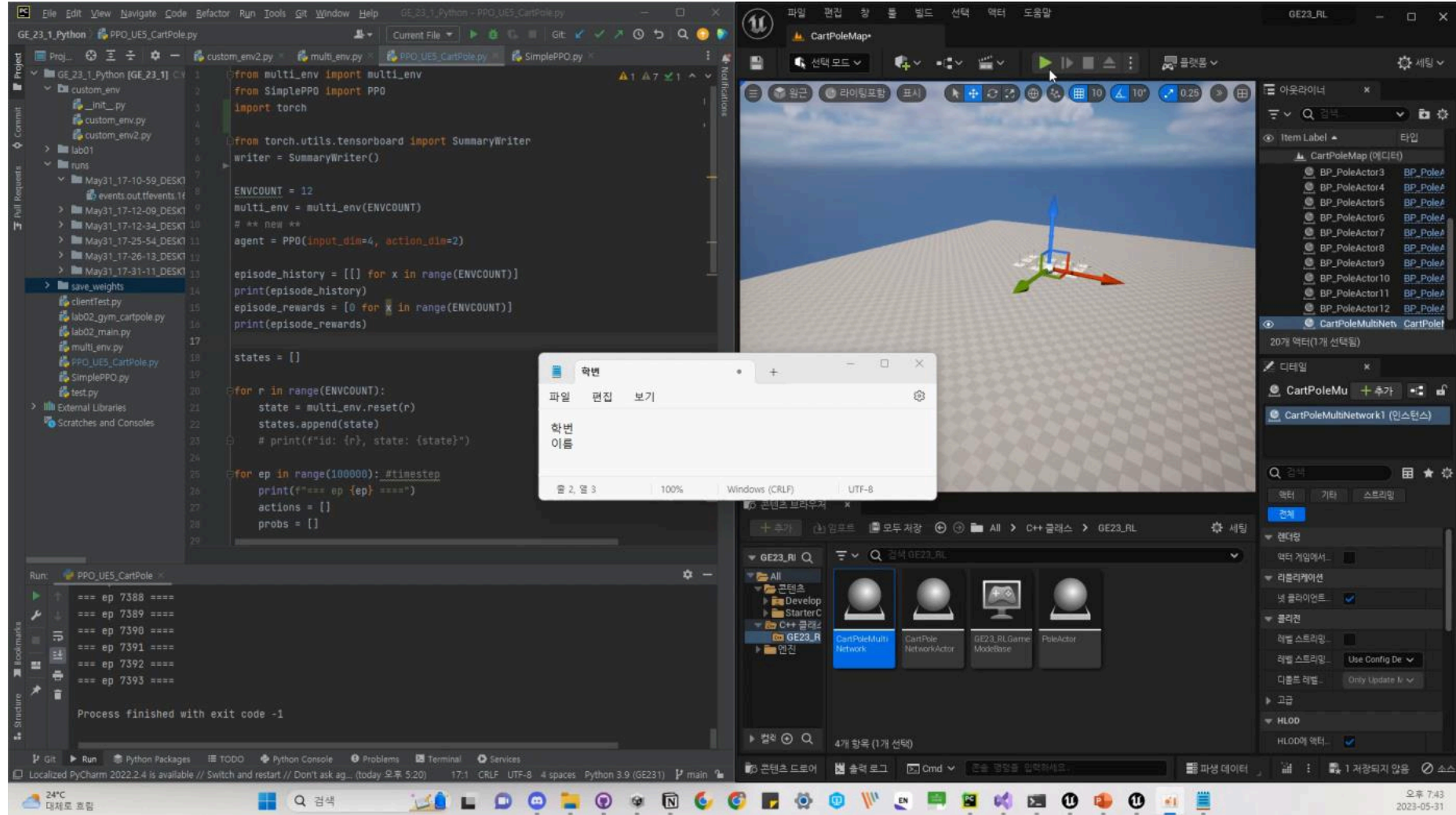
states = n_states
```

Multi Env + PPO

```
if timestep % 300 == 0 and timestep != 0:
    print("===trained===")
    for r in range(ENVCOUNT):
        state = multi_env.reset(r)
    agent.train_net()
```

Submission

- Write your student ID and name, then submit a video of the pole balancing well.
- Note: after launching PyCharm, click the Unreal window again or the connection will drop.



Loading/Saving the Agent & TensorBoard

```
# ** new **
agent = PPO(input_dim=4, action_dim=2)

episode_history = [[] for x in range(ENVCOUNT)]
episode_rewards = [0 for x in range(ENVCOUNT)]
print(episode_history)
print(episode_rewards)

agent.load_state_dict(torch.load(f"./save_weights/PP0.h5"))

states = []

for r in range(ENVCOUNT):
    state = multi_env.reset(r)
    states.append(state)
```

```
probs.clear()
actions.clear()
states.clear()

states = n_states

if timestep % 300 == 0 and timestep != 0:
    print("===trained===")
    for r in range(ENVCOUNT):
        state = multi_env.reset(r)
    agent.train_net()
    torch.save(agent.state_dict(), f"./save_weights/PP0.h5")
```

Loading/Saving the Agent & TensorBoard

- pip install tensorboard
- Log the episode reward of env #0.

```
from multi_env import multi_env
from SimplePPO import PPO
import torch

from torch.utils.tensorboard import SummaryWriter
writer = SummaryWriter()

ENVCOUNT = 12
multi_env = multi_env(ENVCOUNT)
# ** new **
agent = PPO(input_dim=4, action_dim=2)
```

```
# print(actions)
n_states, rewards, dones = multi_env.step(actions)

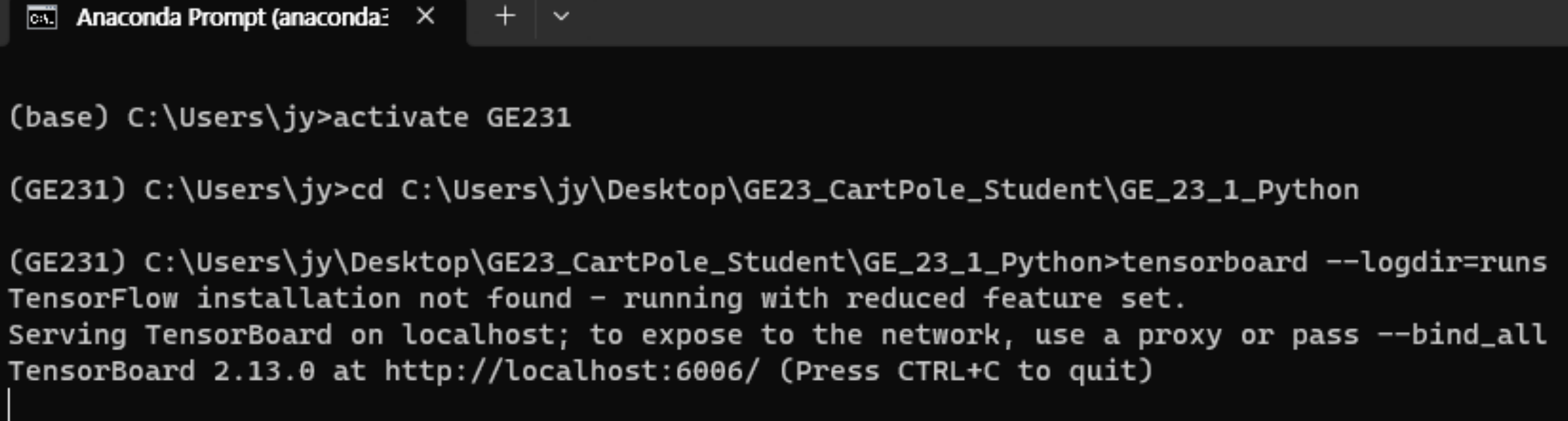
for i in range(ENVCOUNT):
    transition = (states[i], actions[i], rewards[i], n_states[i], probs[i], dones[i])
    # print(transition)
    agent.put_data((states[i], actions[i], rewards[i], n_states[i], probs[i], dones[i]))
    episode_rewards[i] += rewards[i]

    if(dones[i]):
        if(i==0):
            writer.add_scalar("Reward", episode_rewards[i], len(episode_history[i]))
            multi_env.reset(i)
            episode_history[i].append(episode_rewards[i])
            episode_rewards[i] = 0

probs.clear()
actions.clear()
```

Loading/Saving the Agent & TensorBoard

- Move to the directory containing the runs folder, run: `tensorboard --logdir=runs`, then click the link.



```
Anaconda Prompt (anaconda3) C:\Users\jy>activate GE231
(base) C:\Users\jy>activate GE231
(GE231) C:\Users\jy>cd C:\Users\jy\Desktop\GE23_CartPole_Student\GE_23_1_Python
(GE231) C:\Users\jy\Desktop\GE23_CartPole_Student\GE_23_1_Python>tensorboard --logdir=runs
TensorFlow installation not found - running with reduced feature set.
Serving TensorBoard on localhost; to expose to the network, use a proxy or pass --bind_all
TensorBoard 2.13.0 at http://localhost:6006/ (Press CTRL+C to quit)
```

Loading/Saving the Agent & TensorBoard

- The saved reward values appear like this and can be monitored in real time.
- Very handy for checking that long training runs are progressing well.

